



存储系统

- ◆ 存储系统原理
- ◆ 虚拟存储系统
- ◆ Cache存储系统
- ◆ 三级存储系统



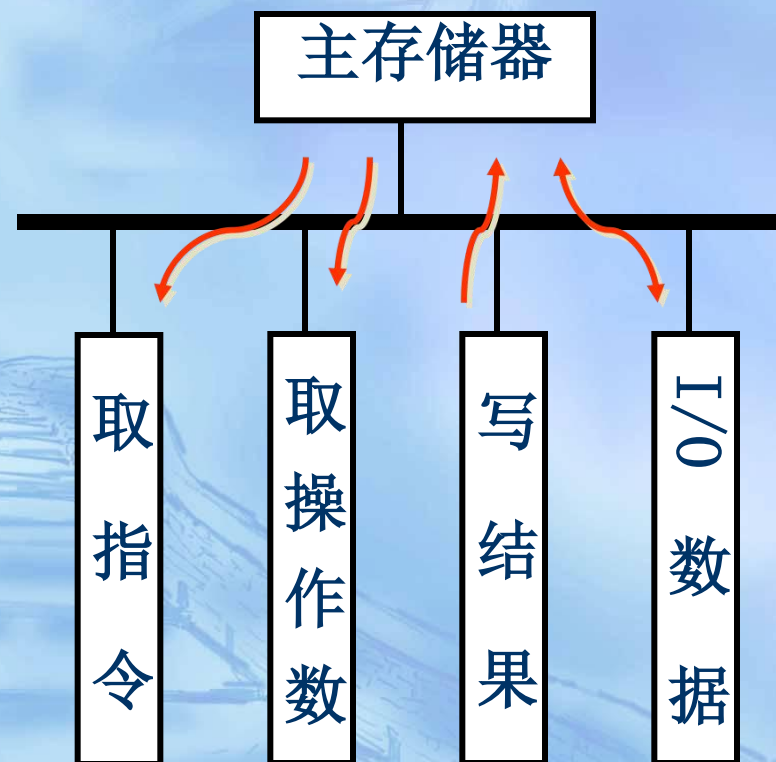
存储系统原理

- ◆ 引入
- ◆ 存储系统的基本概念
- ◆ 存储器的层次结构
- ◆ 存储器的频带平衡



存储器的作用

现代计算机系统都以存储器为中心（不同于以运算器为中心的冯·诺依曼计算机），存储器是各种信息存储和交换的中心。





存储器的性能指标

◆ 存储容量

$S_M = W \cdot l \cdot m$ 。其中： W 为存储体的字长， l 为每个存储体的字数， m 为并行工作的存储体个数。

◆ 存储速度

可以用访问时间 T_A 、存储周期 T_M 和频宽（带宽） B_M 来描述。

◆ 存储价格

可以用总价格 C 或每位价格 c 表示。



存储器的设计

- ◆ **设计目的：** 高速度、大容量、低价格

- ◆ **存在问题**

单靠一种工艺的单一存储器无法同时满足容量、速度和价格三方面的要求。

- 速度越快，每位价格就越高；
- 容量越大，每位价格就越低；
- 容量越大，速度越慢。

- ◆ **解决方法**

使用多种不同工艺的存储器组成**存储系统**，使所有的信息以各种方式分布于不同的存储器上。



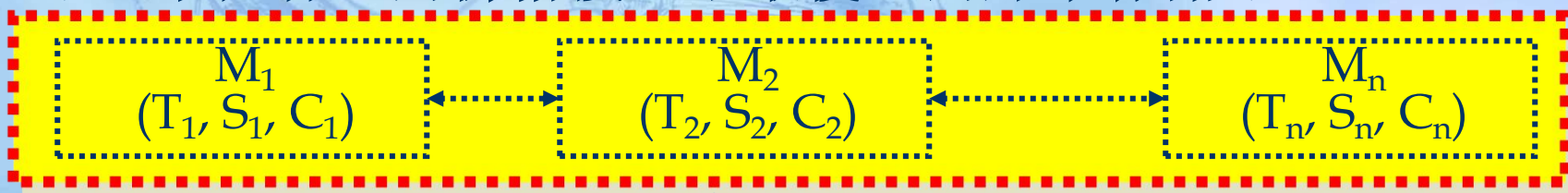
存储系统的概念

- ◆ 存储系统的定义
- ◆ 常用存储系统
- ◆ 存储系统的性能指标
- ◆ 存储系统的设计



存储系统的定义

两个或两个以上**速度**、**容量**和**价格**各不相同的存储器用**硬件**、**软件**或**软硬件**相结合的方法连接起来成为一个**系统**。这个系统对应用程序员**透明**，并且，从应用程序员看它是一个“存储器”，这个“存储器”的速度接近于速度最快的那个存储器，存储容量接近于容量最大的那个存储器，单位容量的价格接近于最便宜的那个存储器。



从外部看

$T \approx \min(T_1, T_2, \dots, T_n)$, 用存储周期表示

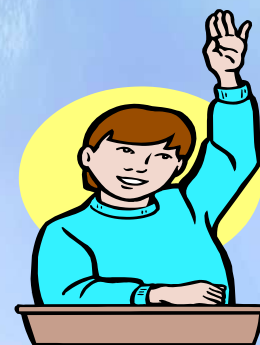
$S \approx \max(S_1, S_2, \dots, S_n)$, 用MB或GB表示

$C \approx \min(C_1, C_2, \dots, C_n)$, 用每位的价格表示



教师和学生

有这么好的事？



当然有！**访存局部性原理**是存储系统设计的基础。



常用存储系统

- ◆ 虚拟存储系统
- ◆ Cache存储系统



虚拟存储系统

◆ 原理

由主存储器和磁盘存储器构成。

◆ 目的

增加存储器的存储容量。

◆ 特点

采用硬件和软件相结合的方法来调度，因此对应用程序员是透明的，对系统程序员是不透明的。

这个存储系统从**应用程序员**看：速度接近主存的速度，容量是虚拟地址空间，每位价格接近磁盘存储器的价格。



主存储器

磁盘存储器





Cache存储系统

◆ 原理

由Cache和主存储器构成。

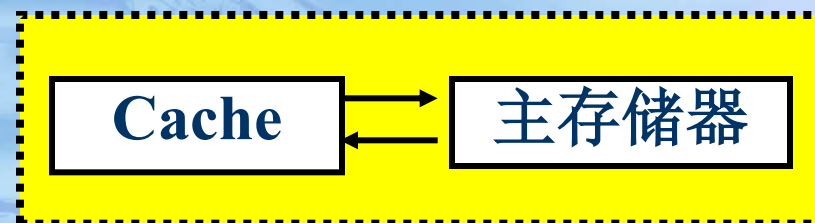
◆ 目的

提高存储器的速度。

◆ 特点

全部用硬件来调度，不仅对应用程序员还是系统程序员都是透明的。

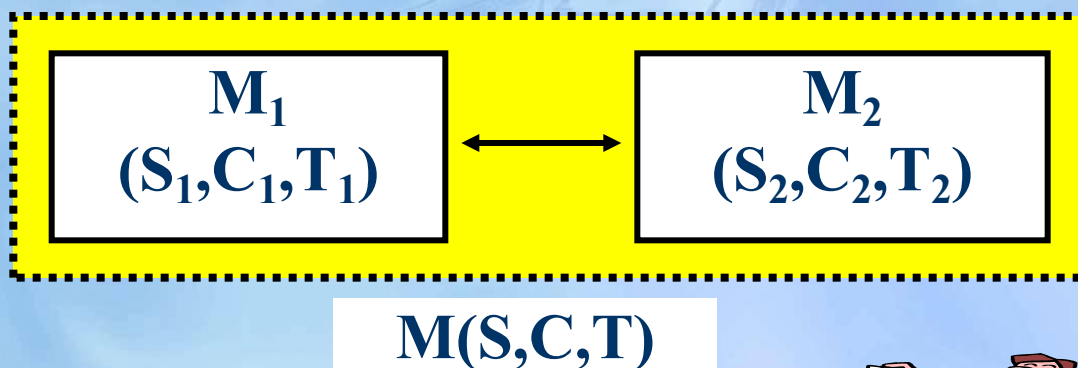
这个存储系统从**系统/应用程序员**看：速度接近Cache的速度，容量是主存的容量，每位价格接近主存的价格。





存储系统的性能指标

- ◆ 存储容量
- ◆ 存储价格
- ◆ 存储速度



为了分析方便，采用由两个存储器M1和M2组成的存储系统M。





存储容量

存储容量接近于 M_2 （即： $S \approx S_2$ ）。对存储系统进行编址的方法有：

- ◆ 可以选择对 M_2 进行编址， M_1 可以不编址或在系统内部编址

例如：Cache存储系统。

- ◆ 为存储系统另外设计一个抽象的地址空间，在系统内部对 M_1 、 M_2 分别编址并将地址映象到这个抽象的地址空间中

例如：虚拟存储系统。



存储价格

整个存储系统的单位容量平均价格为:

$$c = \frac{c1 \cdot S1 + c2 \cdot S2}{S1 + S2}$$

当 $S_2 \gg S_1$ 时, $c \approx c_2$, 但 S_1 与 S_2 不能相差太大, 否则存储系统要达到比较高的性能, 调度起来很困难。



存储速度

◆ 命中率H

在M₁存储器中访问到的概率。

$$H = \frac{N_1}{N_1 + N_2}$$

N₁: M₁的访问次数
N₂: M₂的访问次数

例如：Cache存储系统

◆ 存储系统的平均访问时间T_A

$$T_A = H \cdot T_1 + (1 - H) \cdot T_2 \quad (\text{当} H \rightarrow 1 \text{时}, T_A \rightarrow T_1)$$

提示：访问二级存储系统时，同时访问M₁和M₂，如果在M₁命中，则提前终止M₂的访问；如果在M₁缺失，则继续M₂的访问，将访问的字送交CPU，将访问的字所在块送交M₁。



存储速度

例如：虚拟存储系统

访问二级存储系统时，如果采用先访问M1，当命中时访问时间就是T₁；当访问缺失时，先从M2中将包含要访问的字在内的块/页传送到M1，然后再将访问的字送交CPU。

- ◆ 缺失时的访问时间： $T_2 + T_B + T_1 = T_1 + T_M$
 - T_B：传送一个块/页所需的时间
 - T_M：T_M = T_B + T₂，缺失代价（不命中开销）
- ◆ 存储系统的平均访问时间T_A

$$\begin{aligned} T_A &= H \cdot T_1 + (1 - H) \cdot (T_1 + T_M) \\ &= T_1 + (1 - H) \cdot T_M \end{aligned}$$



存储系统的访问效率

$$\begin{aligned} e &= \frac{T_1}{T_A} = \frac{T_1}{H \cdot T_1 + (1 - H) \cdot T_2} \\ &= \frac{1}{H + (1 - H) \cdot \frac{T_2}{T_1}} \\ &= f\left(H, \frac{T_2}{T_1}\right) \end{aligned}$$

提高存储系统速度的两条途径:

- ◆ 提高命中率H（见例1）
- ◆ 两个存储器的速度不要相差太大（见例3）



例1：不同命中率

问：假设 $T_2=5T_1$ ，在命中率 H 为0.9和0.99两种情况下，分别计算存储系统的访问效率。

答：当 $H=0.9$ 时：

$$e = \frac{1}{0.9 + (1 - 0.9) \cdot 5} = 0.72$$

当 $H=0.99$ 时：

$$e = \frac{1}{0.99 + (1 - 0.99) \cdot 5} = 0.96$$



采用预取技术提高命中率

◆ 思想

不命中时，把 M_2 存储器中相邻几个单元组成的一个数据块都取出来送入 M_1 存储器中。

◆ 命中率

$$H' = 1 - \frac{1-H}{n} = \frac{H+n-1}{n}$$

(见例2)

其中：**H'**是采用预取技术后的命中率；

H是原来的命中率；

n为数据块大小与数据重复使用次数的乘积。



例2：预取技术

问：在一个虚拟存储系统中， $T_2 = 10^5 T_1$ ，原来的命中率只有**0.8**，如果访问磁盘存储器的数据块大小为**4K**字，并要求访问效率不低于**0.9**，计算数据在主存储器中的重复利用率至少为多少？

答：假设数据在主存储器中的重复利用率为**m**，根据前面的给出关系：

$$0.9 = \frac{1}{H' + (1 - H') \cdot 10^5}, \quad H' = \frac{0.8 + 4096m - 1}{4096m}$$

解之得：m=44



例3：两个存储器的速度相差太大

问：在虚拟存储系统中，两级存储器的速度相差特别悬殊 $T_2=10^5T_1$ 。如果要使访问效率 $e=0.9$ ，问需要有多高的命中率？

答：

$$0.9 = \frac{1}{H + (1 - H) \cdot 10^5}$$

解之得：

$$H = 0.999998888877777... \approx 0.999999$$



存储系统的设计

◆ 设计原则

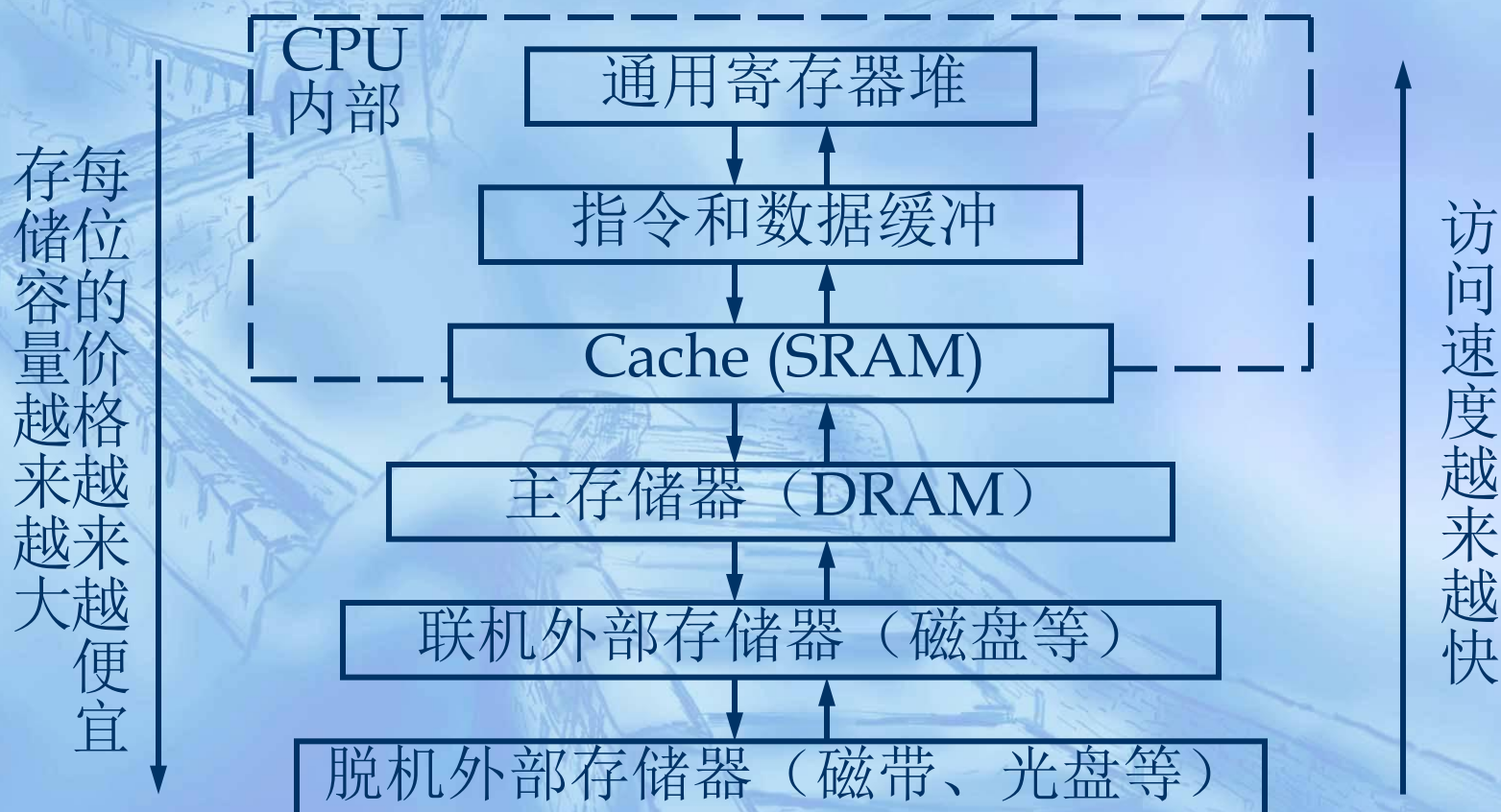
- 相邻级的容量差、速度差较大；(减少C)
- 存储系统具有较高的命中率；(减少T)
- 存储系统的辅助软、硬件开销较小。

◆ 涉及问题

- 映象规则：块从低层调入高层时放在何位置；
- 查找算法：如何在本系统中查找需访问的块；
- 替换算法：发生失效时，替换哪个块；
- 写策略：进行写访问时，应进行那些操作。



存储器的层次结构





存储器的频带平衡

◆ 问题

计算机中各级存储器频带应该达到平衡，即：存储器的速度应能跟得上系统的需要。

◆ 方法

- 多个存储器并行，采用并行/交叉访问等方法提高存储器的访问速度（并行存储器）；
- 设置各种缓冲存储器；
- 采用存储系统，特别是**Cache**存储系统。



并行存储器

- ◆ 并行访问存储器
- ◆ 交叉访问存储器



并行访问存储器

◆ 思想

增加存储器的字长，例如：把 m 字 w 位的存储器（单体单字存储器）改变成为 m/n 字 $n \times w$ 位的存储器（单体多字存储器），见后图。

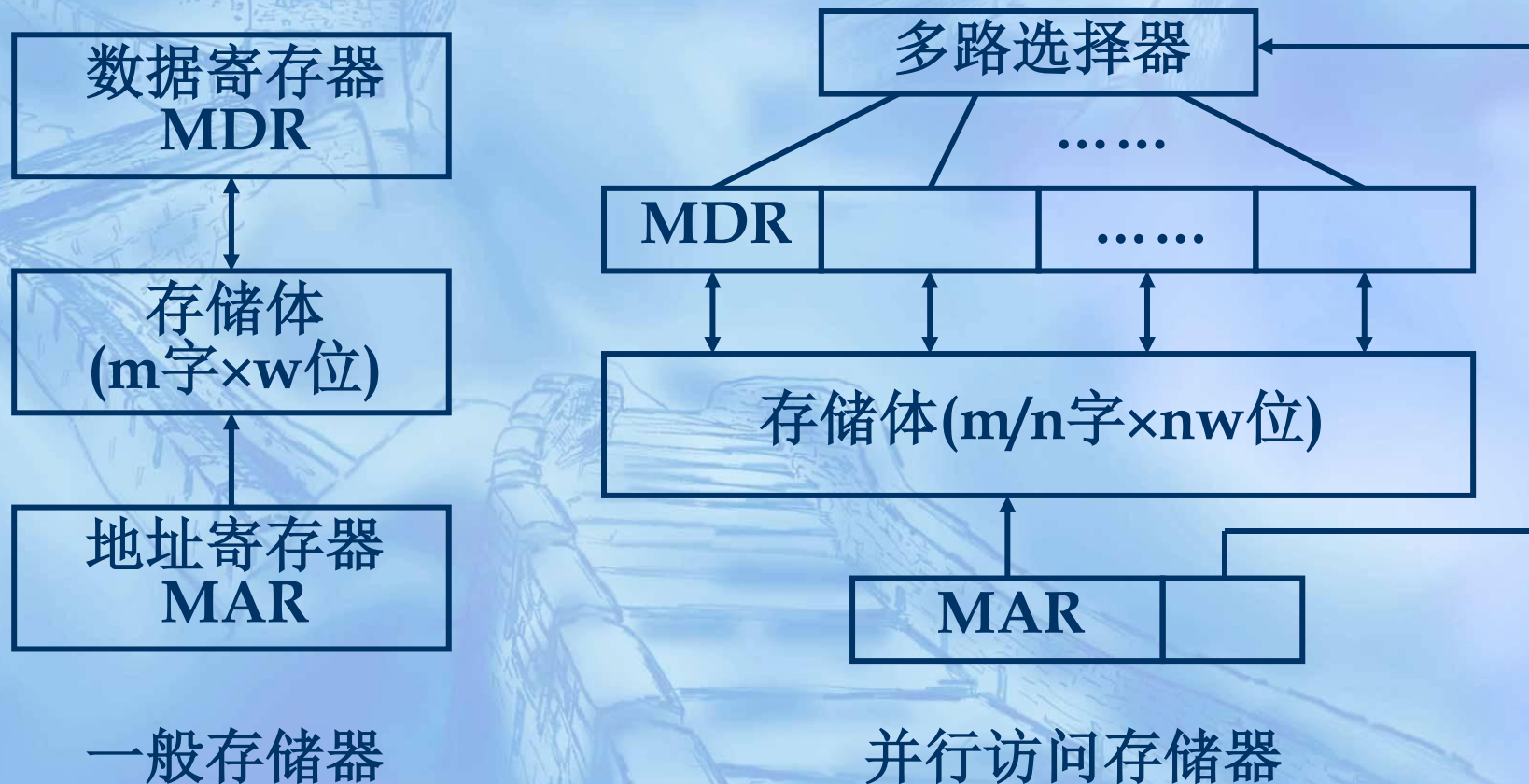
◆ 特点

优点：实现简单、容易。

缺点：访问的冲突大（取指令冲突、读操作数冲突、写数据冲突和读写冲突）。



图示并行访问存储器



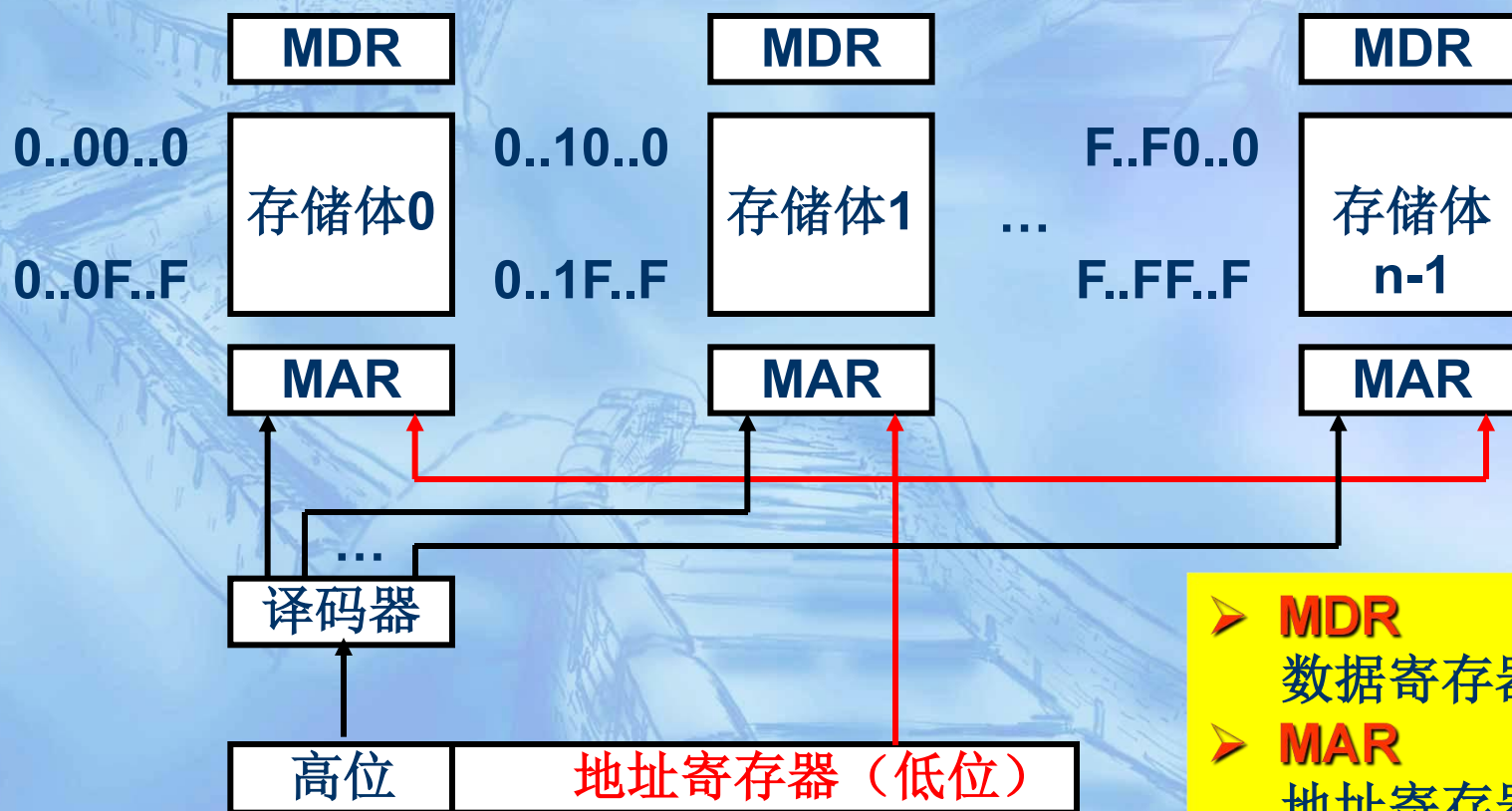


交叉访问存储器

- ◆ 地址码高位交叉
- ◆ 地址码低位交叉



地址码高位交叉访问 存储器



- **MDR** 数据寄存器
- **MAR** 地址寄存器



地 址

存储器某单元的地址为: $A = m \times k + j$

m: 为每个存储体的容量（地址码的低 $\log_2 m$ 位作为存储体的体内地址，而且每个存储体都相同）。

k: 为存储体的编号， $k=0,1,2,\dots,n-1$ （其中 n 为组成存储器的存储体个数的总和，地址码的高 $\log_2 n$ 位作为一个译码器的输入）

j: 为各个存储体的体内地址， $j=0,1,2,\dots,m-1$

如果已知地址 A ，则：

存储器的体内地址 A_j 的计算公式为: $A_j = A \bmod m$

存储器的体号 A_k 的计算公式为: $A_k = \lfloor A/m \rfloor$



目的

◆ 目的

扩大存储器容量。

◆ 例子

目前，大部分计算机系统中所采用的模块化主存储器通常都是采用高位交叉编址方法实现。

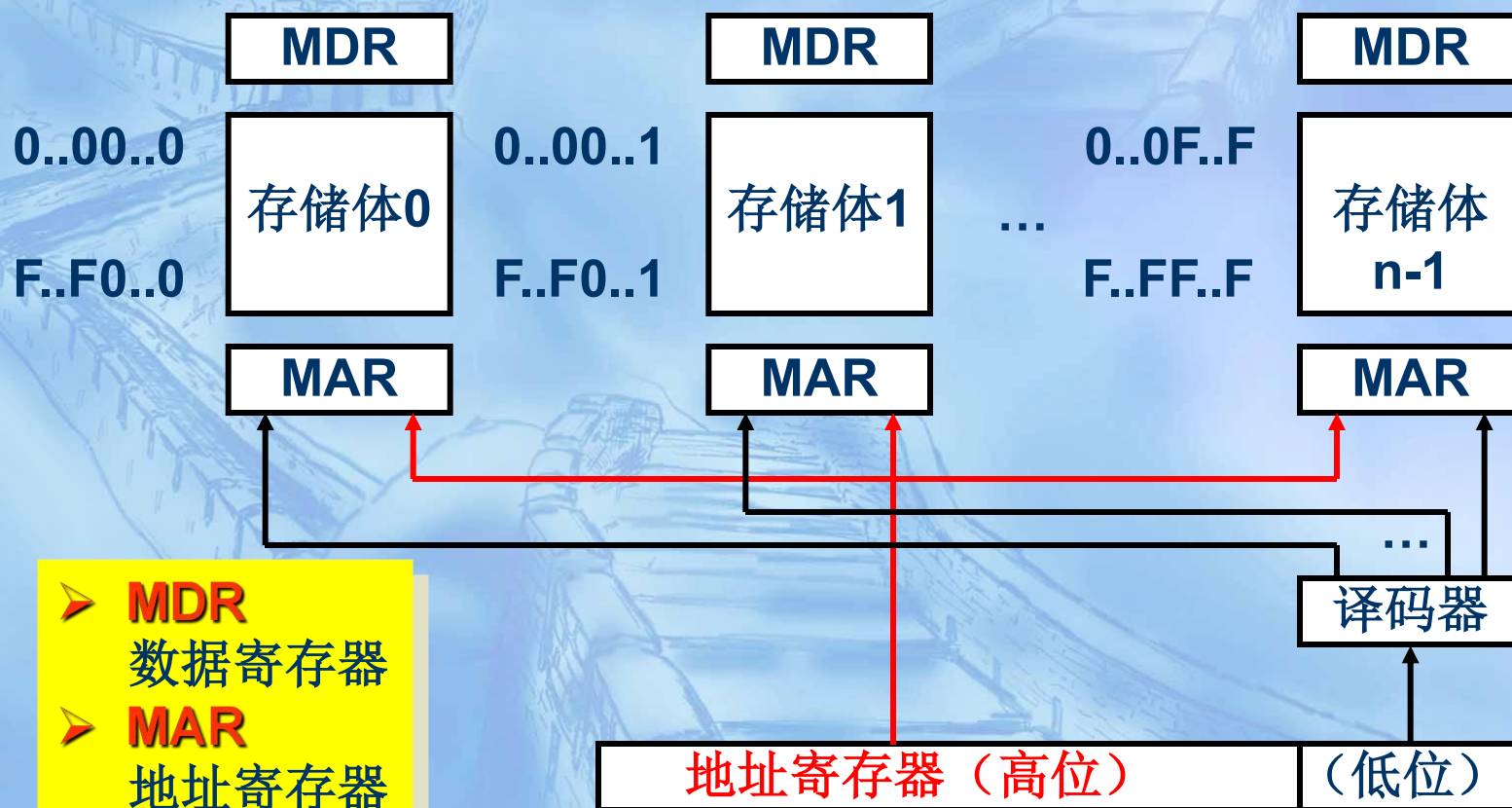
◆ 应用

在单任务系统中：可用于扩大存储器容量，且扩充性好。

在多任务或多用户系统中：可以通过把不同的任务分配给不同的存储体完成来提高存储器的访问速度。



地址码低位交叉访问 存储器





地 址

存储器某单元的地址为: $A = n \times j + k$

m: 为每个存储体的容量（地址码的高 $\log_2 m$ 位作为存储体的体内地址，而且每个存储体都相同）。

k: 为存储体的编号， $k=0,1,2,\dots,n-1$ （其中 n 为组成存储器的存储体个数的总和，地址码的低 $\log_2 n$ 位作为一个译码器的输入）

j: 为各个存储体的体内地址， $j=0,1,2,\dots,m-1$

如果已知地址 A ，则：

存储器的体内地址 A_j 的计算公式为: $A_j = \lfloor A/n \rfloor$

存储器的体号 A_k 的计算公式为: $A_k = A \bmod n$



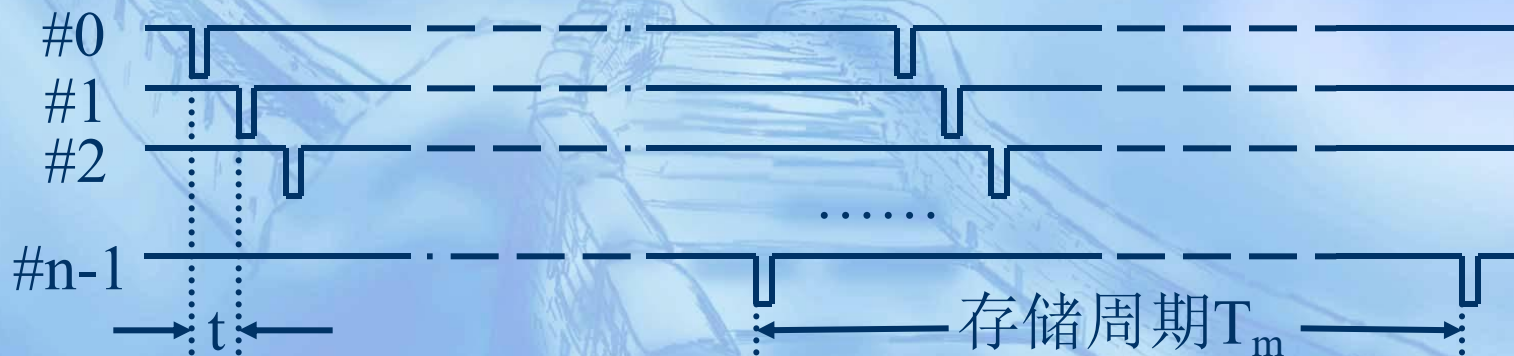
目的

◆ 目的

提高存储器访问速度。

◆ 实现

为此在一个存储周期内， n 个存储体必须同时或分时启动，实际上是一种采用流水线方式工作的并行存储器。



$$\text{启动间隔 } t = \lfloor T_m / n \rfloor$$



问题

◆ 问题

主存储器的速度不是随存储体的个数的增加而线性增加。例如：在有的大型计算机中采用**32**个存储体低位交叉来构成主存储器，但是主存储器的速度只比单个存储体高**10**倍左右。

◆ 原因

存在访问冲突，产生冲突的根源主要有二：程序中有转移指令和数据的随机性。

◆ 解决

设计一种无访问冲突的存储器。

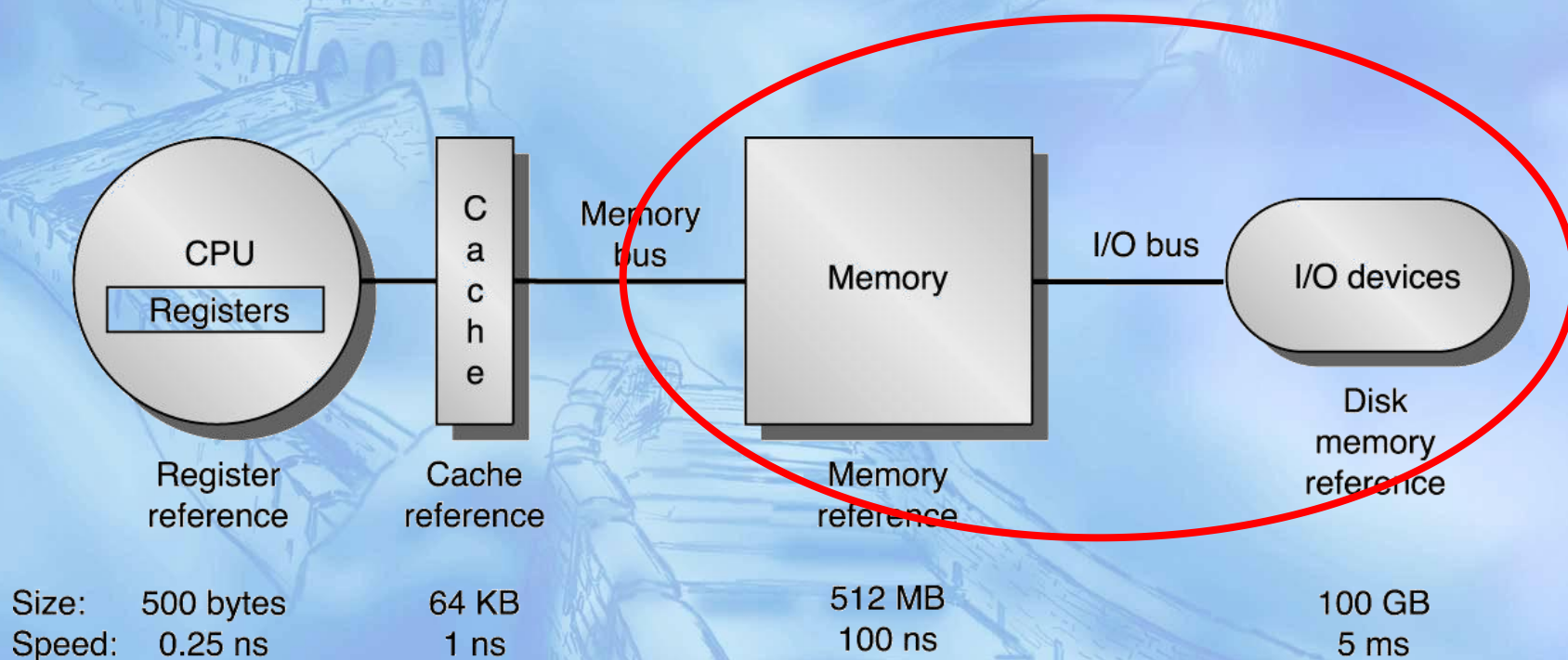


虚拟存储系统

虚拟存储系统也称虚拟存储器、虚拟存储体系，1961年英国曼彻斯特大学Kilbrn等人提出，70年代广泛地应用于大中型计算机系统中，目前许多微型机也开始使用虚拟存储器。



虚拟存储系统

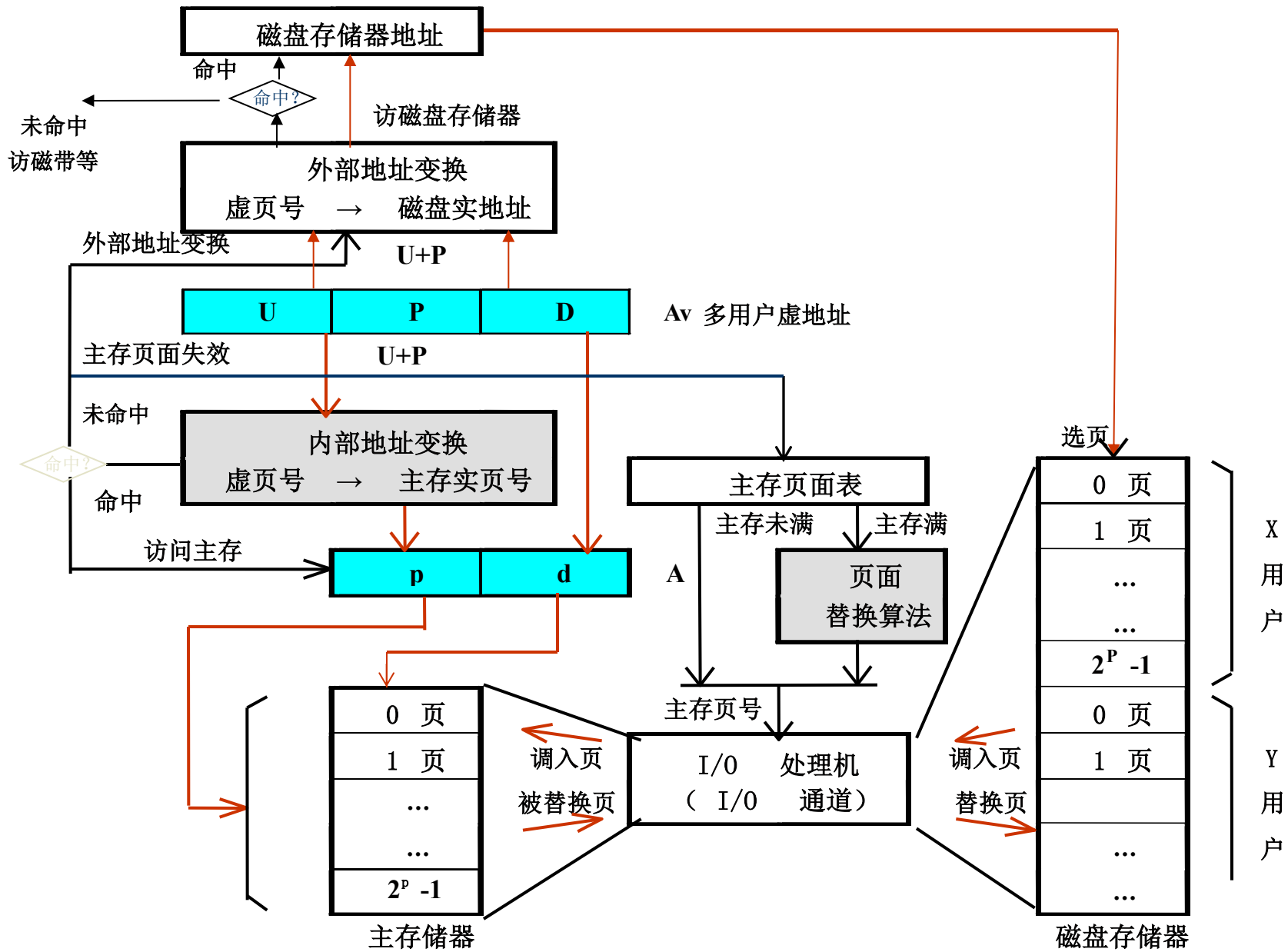




虚拟存储系统

- ◆ 虚拟存储系统的工作原理
- ◆ 地址的映象和变换方法
- ◆ 加快内部地址变换速度的方法
- ◆ 页面替换算法
- ◆ 提高主存命中率的方法

页式虚拟存储器工作原理





基本概念

- ◆ **三个地址空间**

虚拟地址空间、主存地址空间和辅存地址空间。

- ◆ **三个地址**

虚拟地址、主存地址和辅存地址。

- ◆ **地址映象**

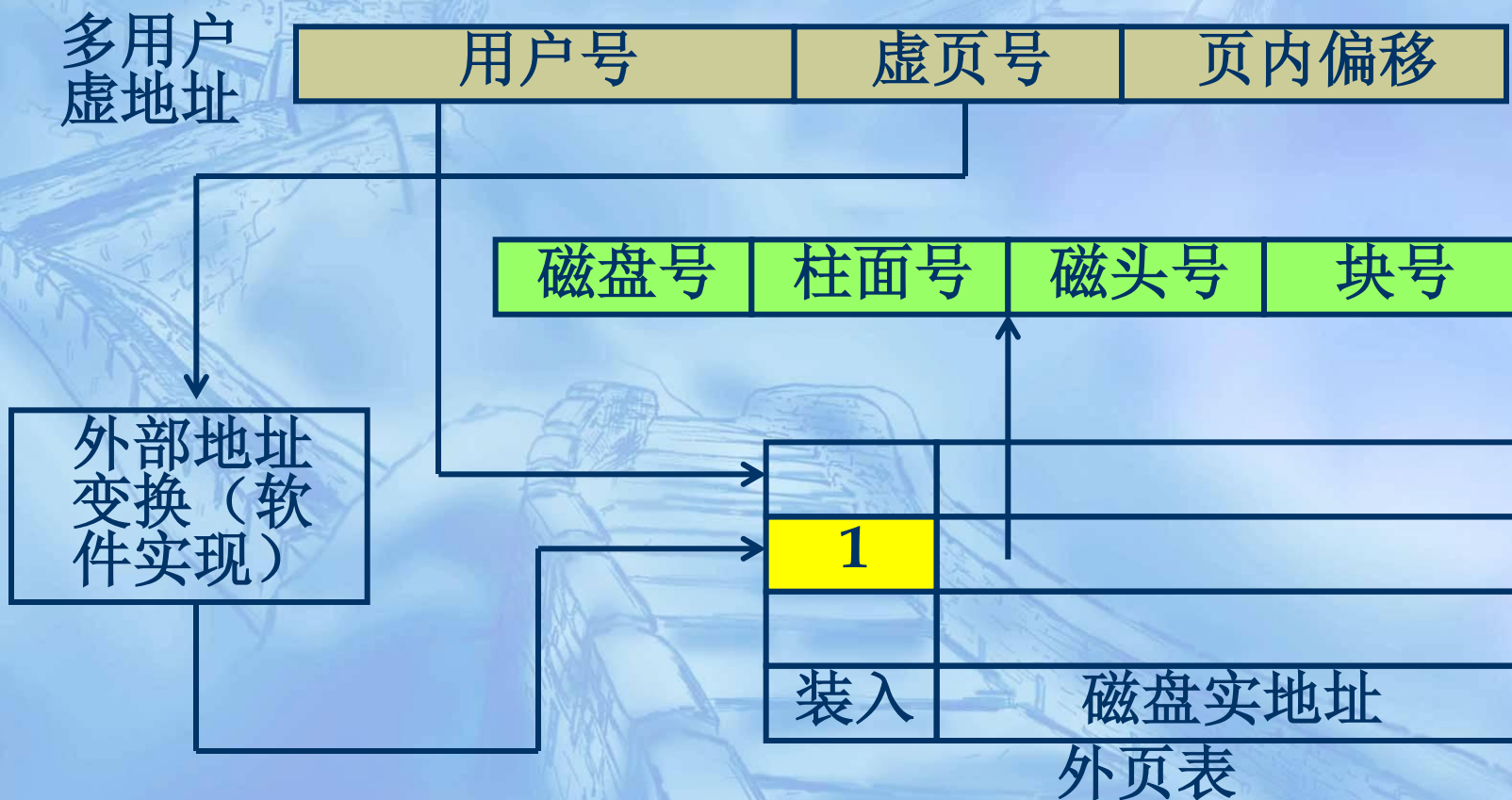
把虚拟地址空间映象到主存地址空间。

- ◆ **地址变换**

在程序运行时，把虚地址变换成主存地址（**内部地址变换**）或辅存地址（**外部地址变换**）。



外部地址变换





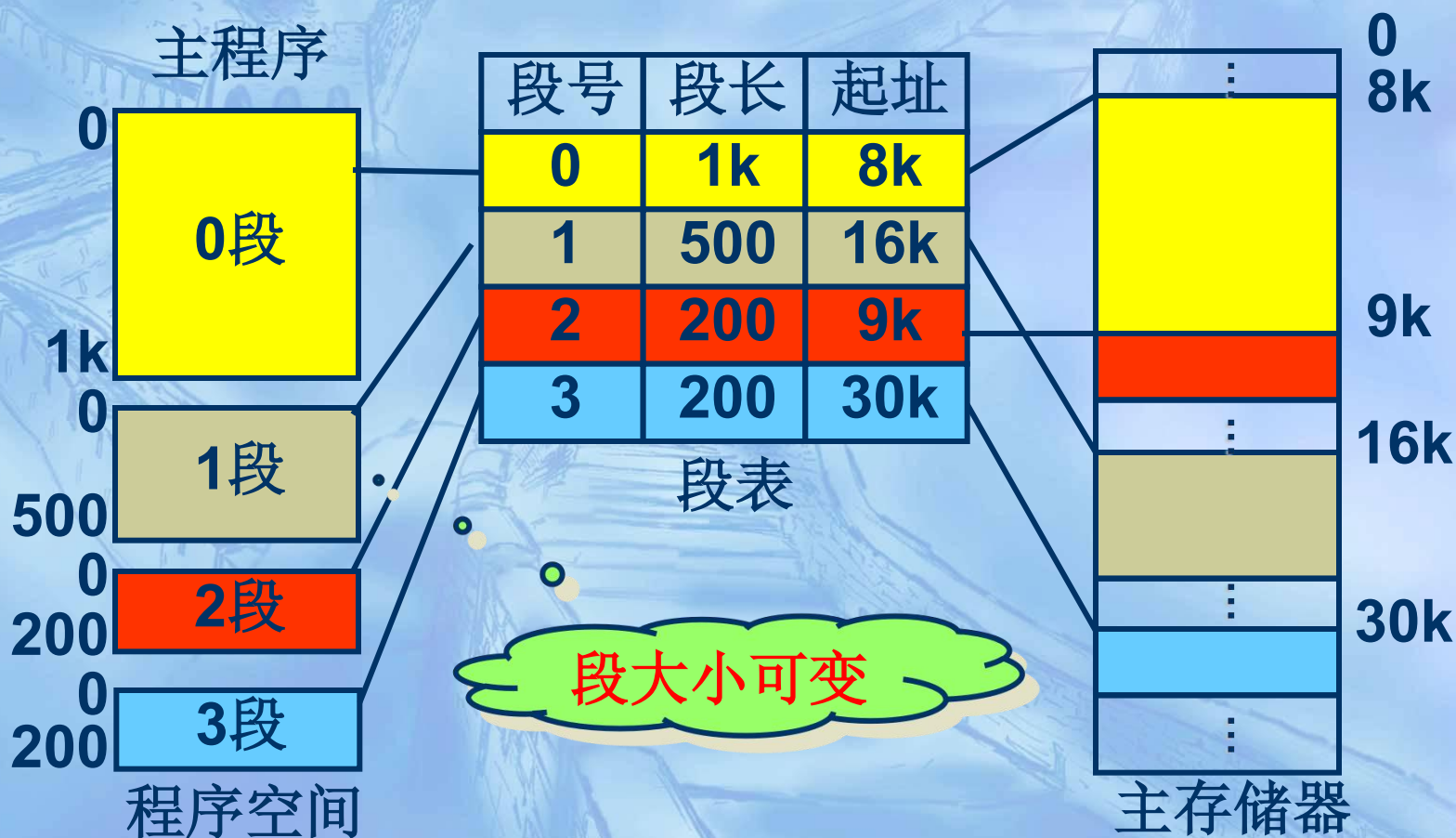
主要内容

根据所采用的地址映象和变换方法（内部地址变换）不同，有三种不同类型的虚拟存储器：

- ◆ 段式虚拟存储器
- ◆ 页式虚拟存储器
- ◆ 段页式虚拟存储器

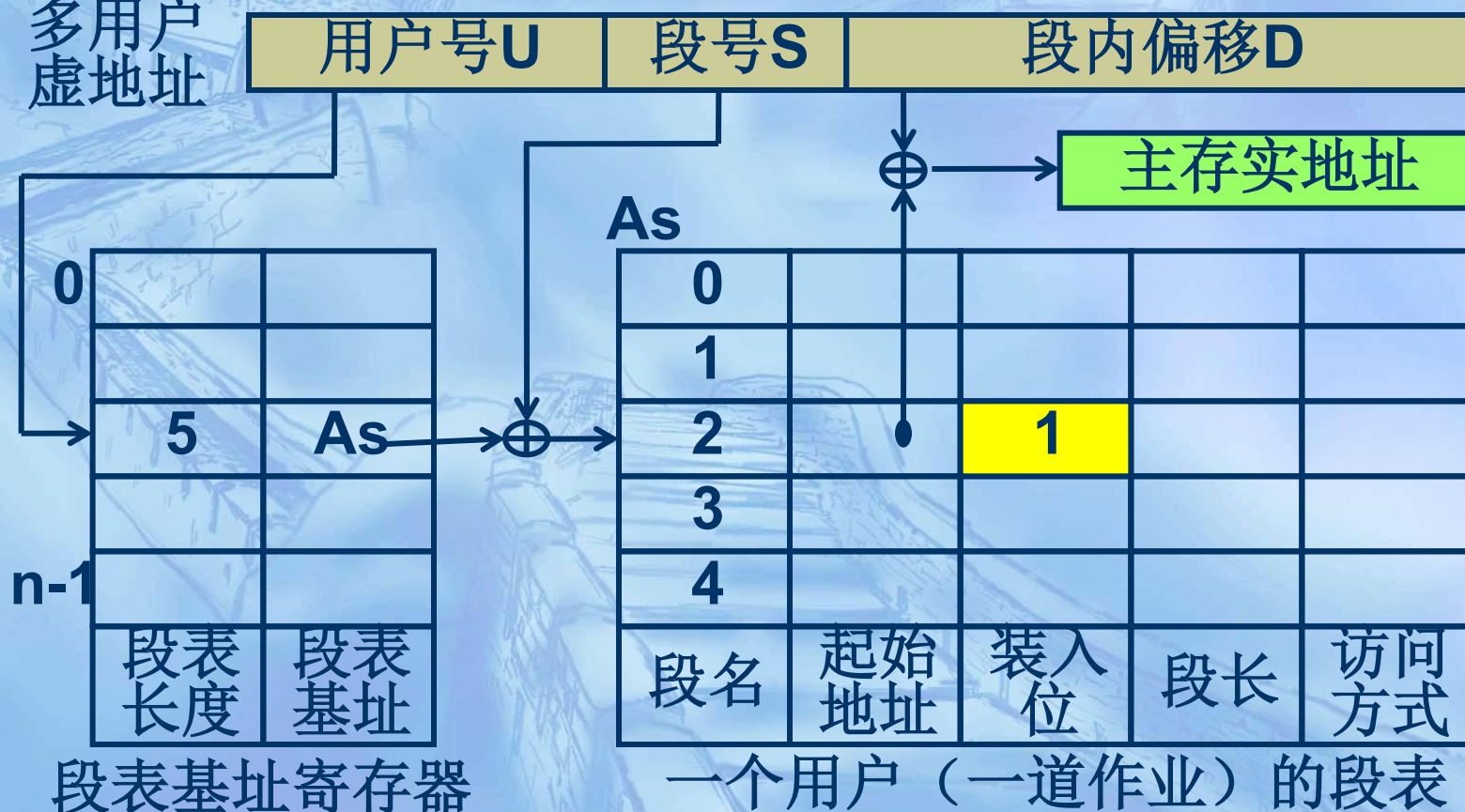


段式虚拟存储器的地址映象





多用户 虚地址





段式虚拟存储器的特点

◆ 优点

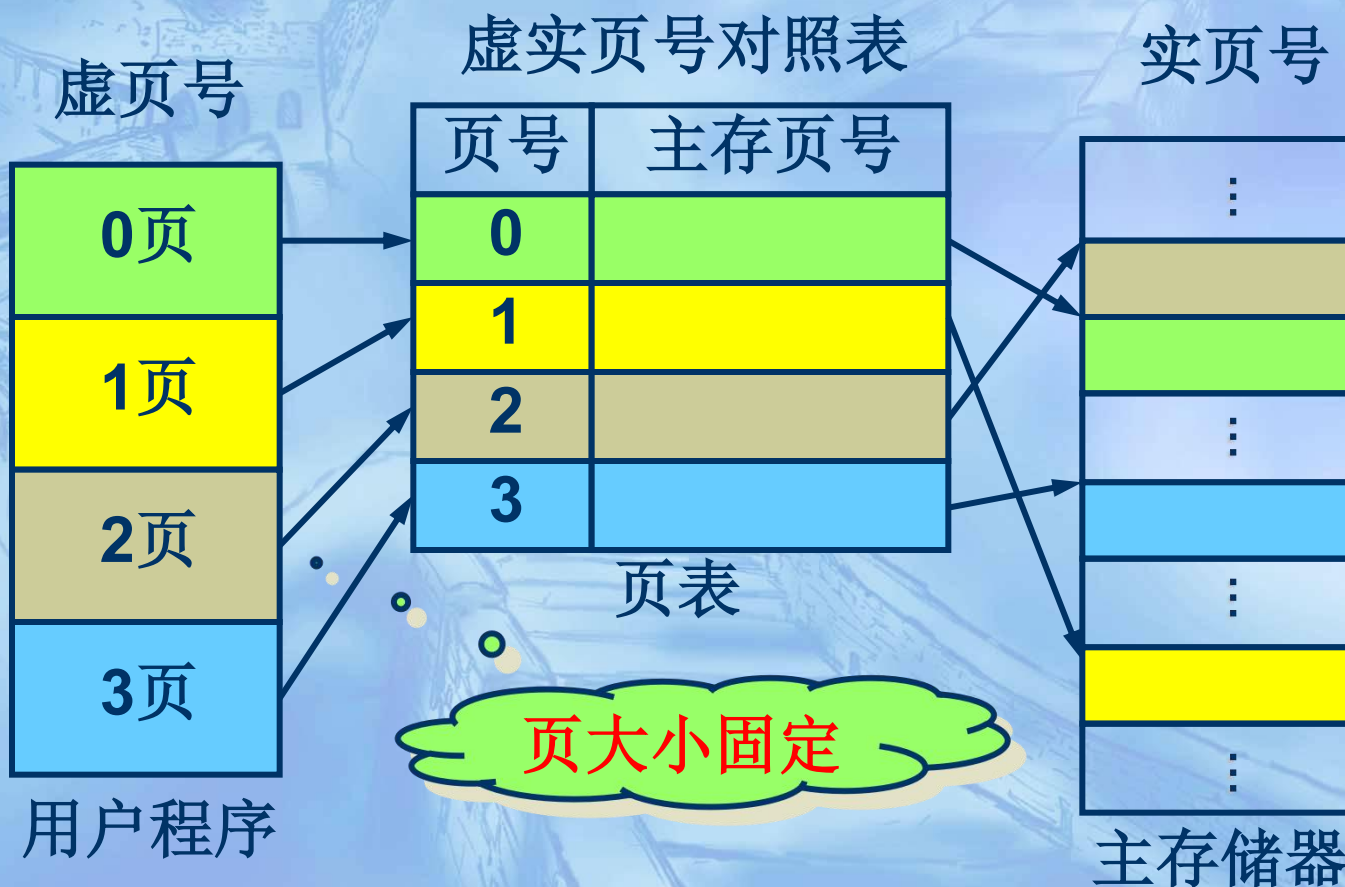
- 程序的模块化性能好
- 便于程序和数据共享
- 程序的动态链接和调度比较容易
- 便于实现信息保护

◆ 缺点

- 地址变换所花费的时间比较长，做两次加法运算
- 主存储器的利用率往往比较低
- 对辅存（磁盘存储器）的管理比较困难

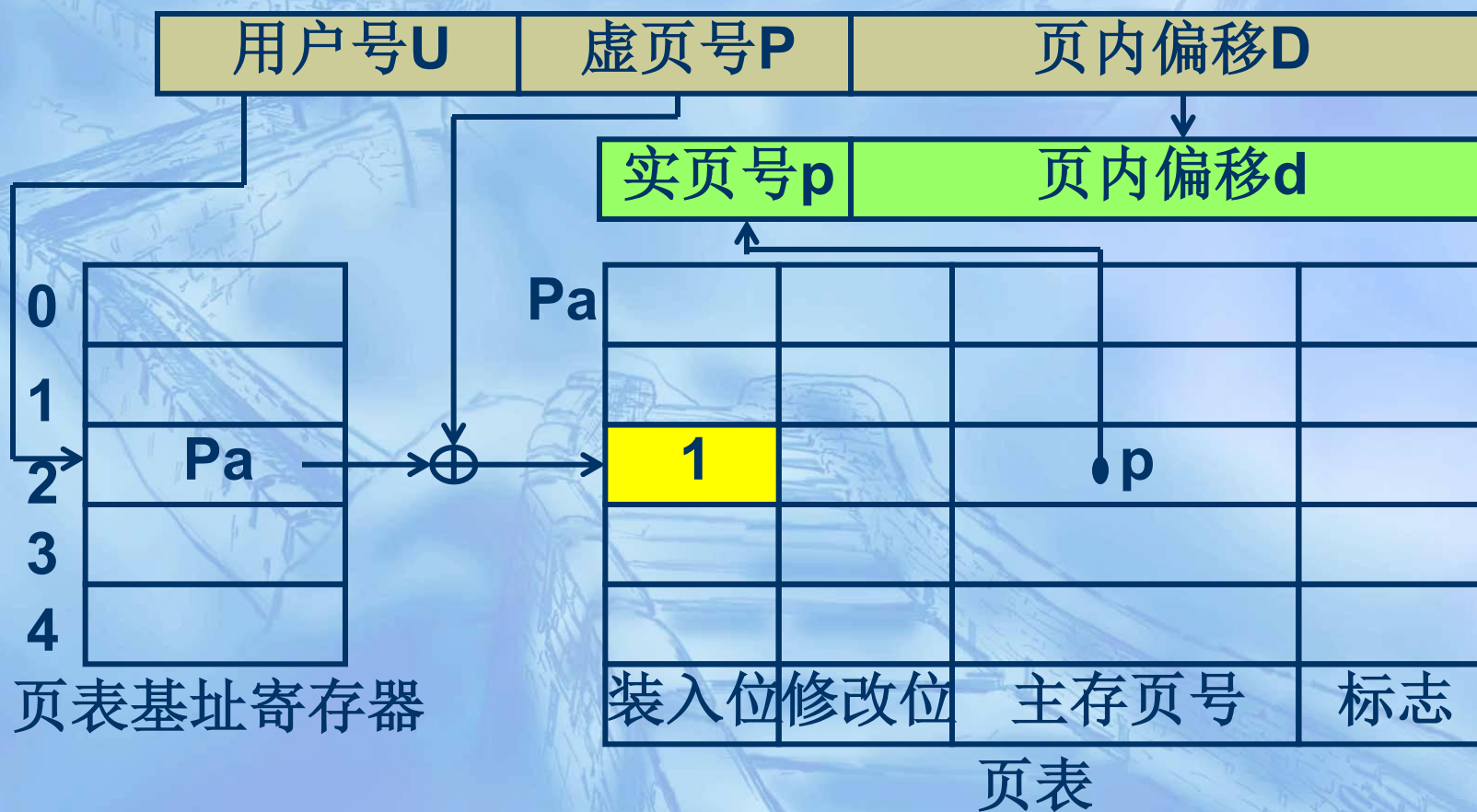


页式虚拟存储器的地址映象





页式虚拟存储器的地址变换





页式虚拟存储器的特点

◆ 优点

- 主存储器的利用率比较高;
- 页表相对比较简单;
- 地址变换的速度比较快;
- 对磁盘的管理比较容易。

◆ 缺点

- 程序的模块化性能不好;
- 页表很长, 需要占用很大的存储空间。



段页式虚拟存储器的地址映象

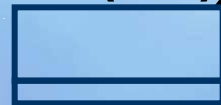
0段(12K)



1段(10K)



2段(5K)

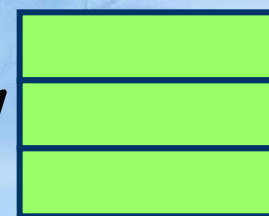


用户程序

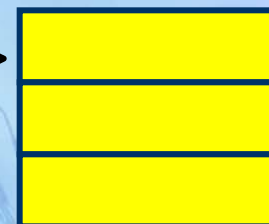
每页4KB

页表长度	页表地址
3	
3	
2	

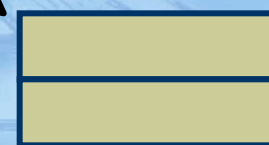
段表



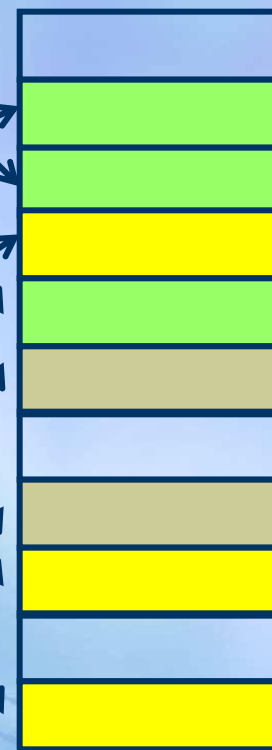
0段页表



1段页表



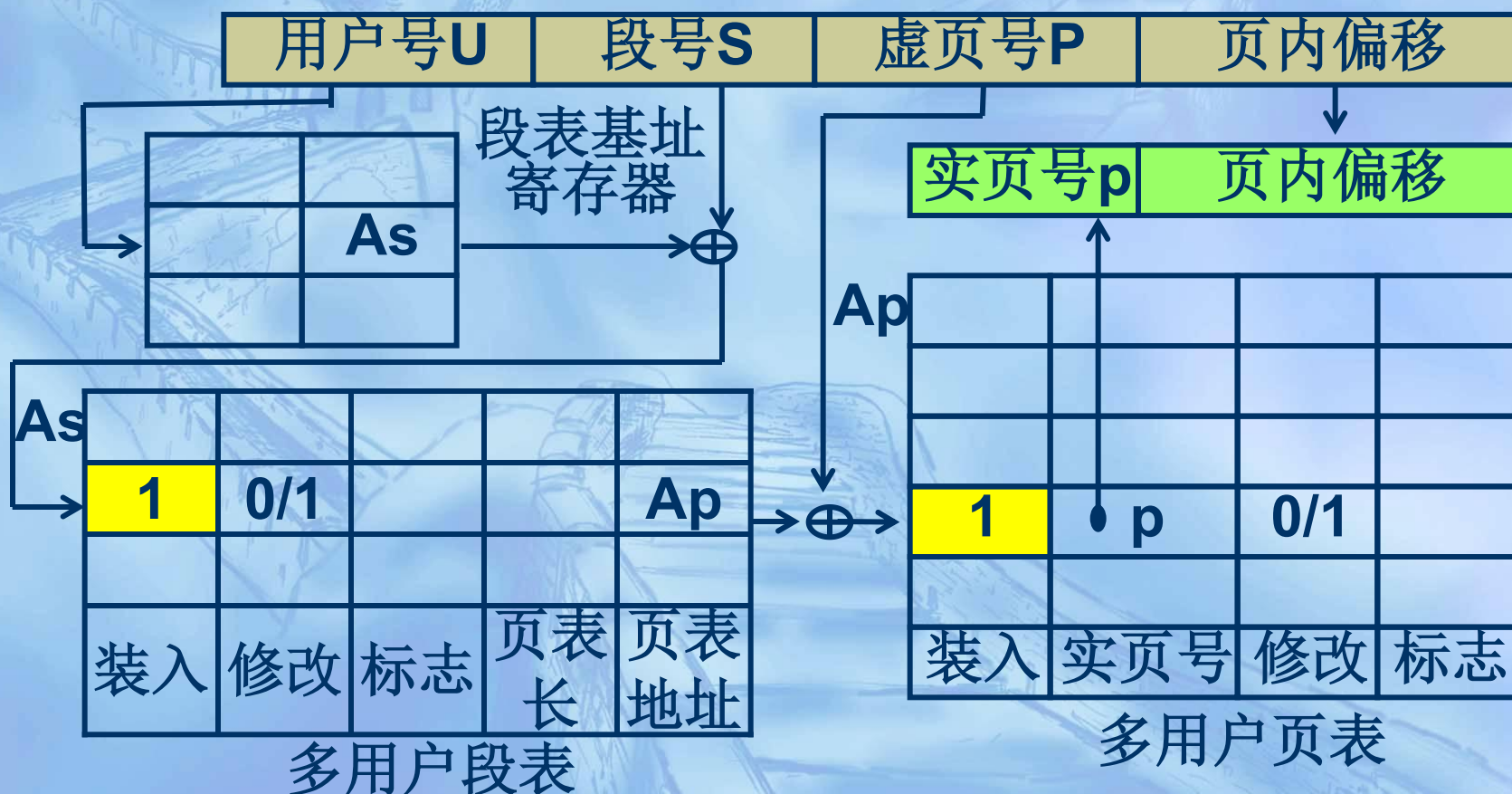
2段页表



主存储器



段页式虚拟存储器的地址变换





段页式虚拟存储器的特点

段页式虚拟存储器一方面具有段式虚拟存储器的主要优点，另一方面具有页式虚拟存储器的主要优点。



加快内部地址变换速度的方法

- ◆ 引入
- ◆ 目录表
- ◆ 快慢表
- ◆ 散列函数
- ◆ 举例



引入

在段/页式虚拟存储器中，要访问主存必须先查段/页表，在段页式虚拟存储器中，既要查段表也要查页表。如果段、页表都在主存中，则包括访问主存本身这一次在内，虚拟存储器的访问速度将要降低**2至3倍**。

因此要想使虚拟存储器的速度接近主存的速度，必须加快查表的速度。



下面以页式虚拟存储器
为例进行介绍

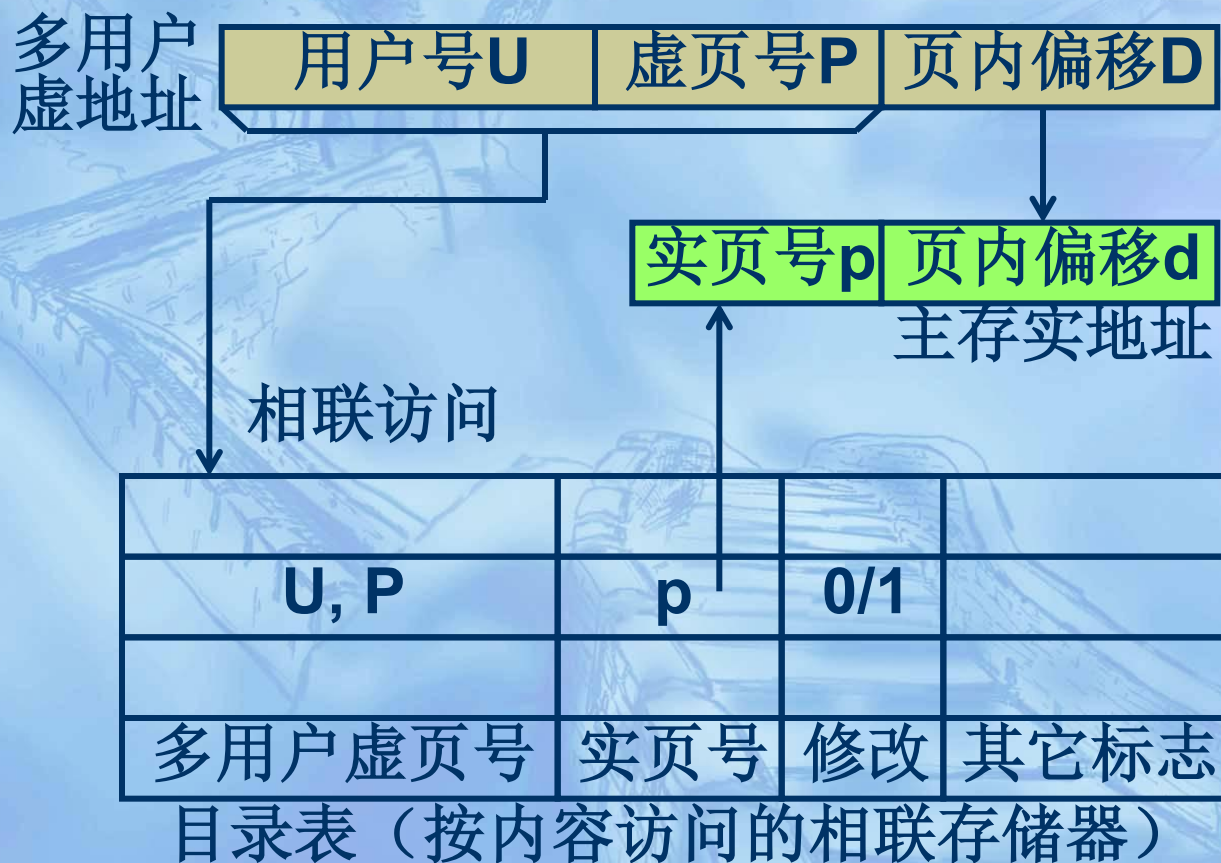


目录表的基本思想

压缩页表的存储容量（因为虚存页面数远大于主存页面数），用一个小容量高速存储器存放页表，从而加快页表的查表速度。



目录表的实现



◆ 页表压缩

页表中只保留已装入主存的那些页。

◆ 高速存储器

采用按内容访问的相联存储器。



目录表的特点

◆ 优点

与页表放在主存中相比，查表速度快。

◆ 缺点

可扩展性比较差，且主存储器容量增加时，目录表的造价高，速度降低。



快慢表的基本思想

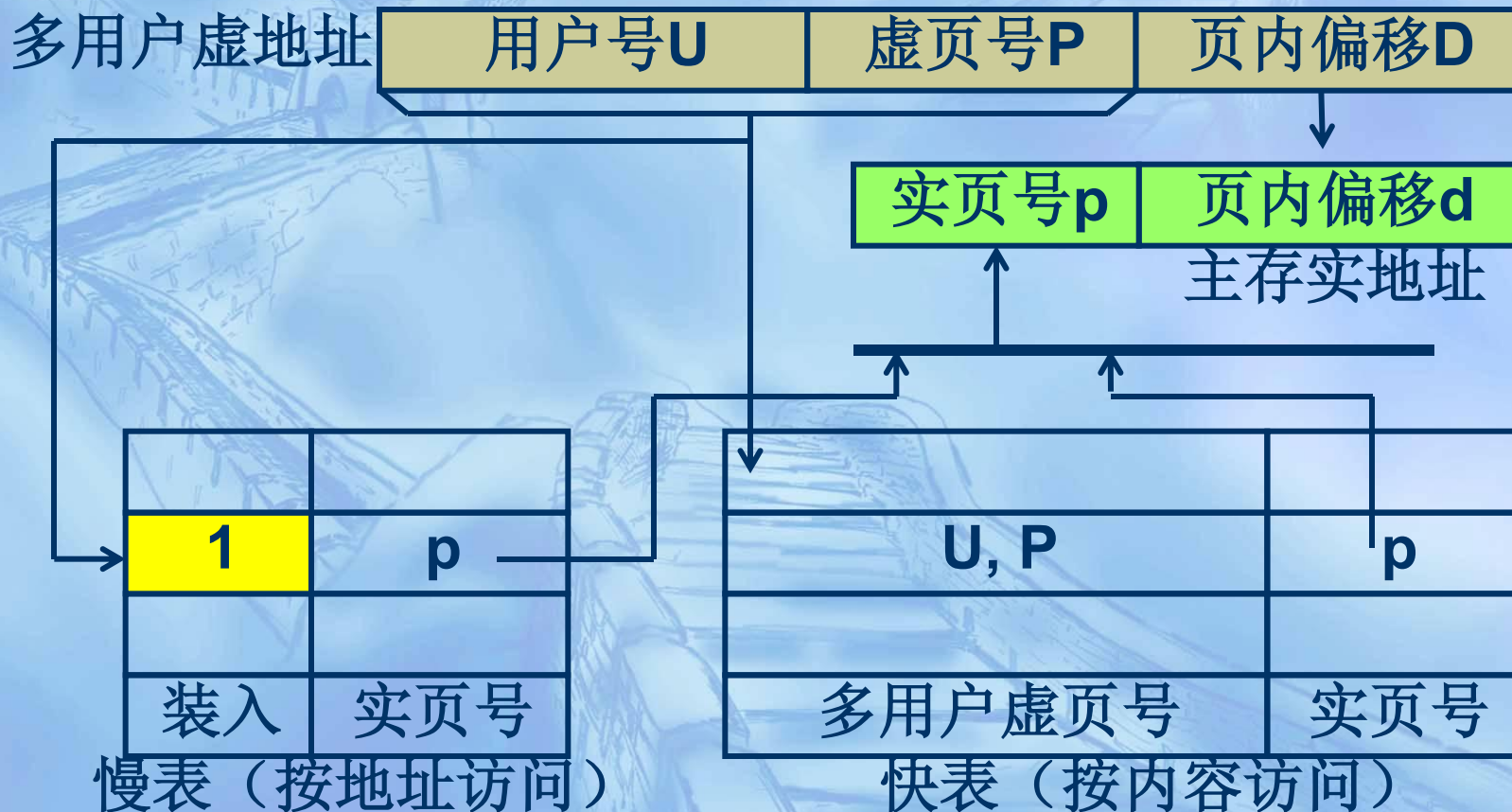
根据局部性原理，将页表分为快表和慢表。快表TLB(Translation Lookaside Buffer)由小容量（几~几十个字）、高速硬件实现，采用相联方式访问，存放最近用到的页表信息。当快表中查不到时，再从存放在主存储器中的慢表中查找。

快表与慢表也构成了一个两级存储系统。





快慢表的实现



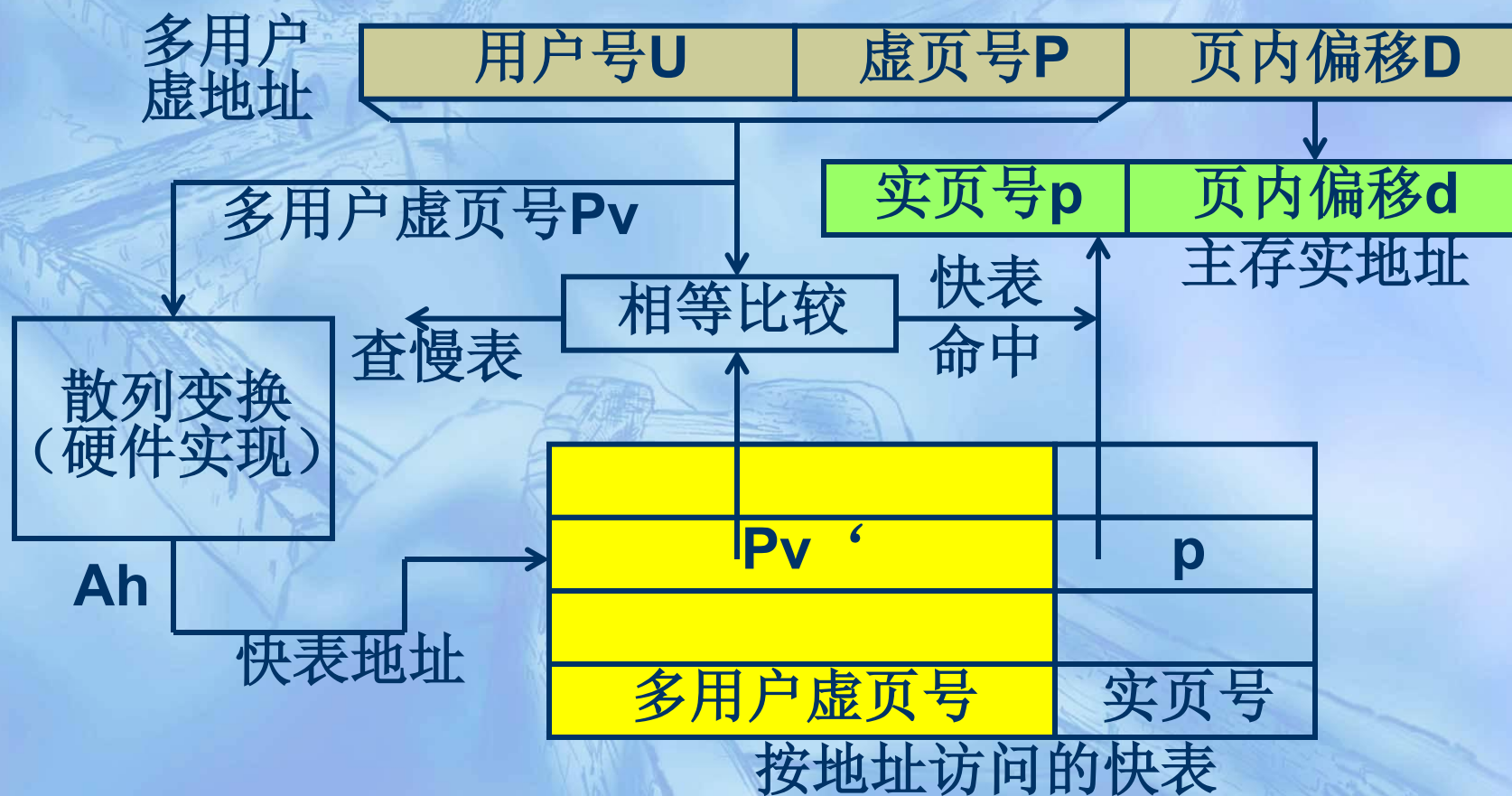


散列函数的基本思想

将快表的按内容相联访问变成按地址访问，从而可以加大快表容量。为提高快表的查找速率采用散列查找法，散列（Hash）函数为：快表地址=H(多用户虚页号)，为避免散列冲突采用相等比较器。



散列函数的实现



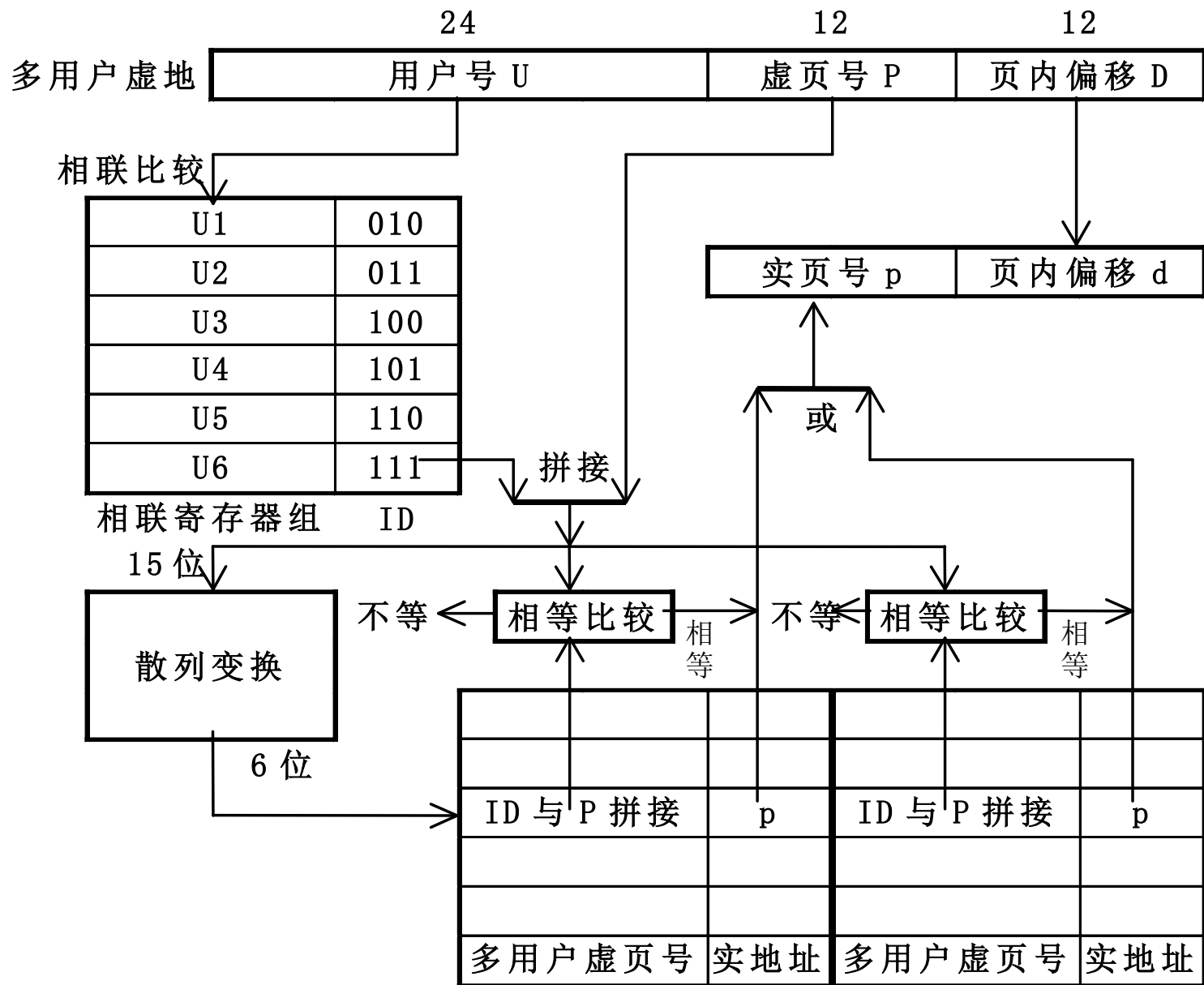


举例：IBM 370/168

IBM 370/168计算机的虚拟存储器快表结构及地址变换过程见后图。虚拟地址长**48**位，页面大小为**4KB**，每个用户最多占用**4K**个页面，最多允许**16M**个用户，但同时上机的用户数一般不超过**6**个。

采用了两项新的措施：

- ◆ 采用两个相等比较器
- ◆ 用相联寄存器组把**24**位用户号**U**压缩成**3**位



快表（按地址访问，有两组）

IBM370/168 计算机的虚拟存储器快表结构





页面替换算法

- ◆ 概述
- ◆ 页面替换算法
 - 随机算法 (RAND 算法)
 - 先进先出算法 (FIFO算法)
 - 近期最少使用算法 (LFU算法)
 - 最久没有使用算法 (LRU算法)
 - 最优替换算法 (OPT算法)
- ◆ 堆栈型算法



概述

◆ 为什么需要页面替换？

当发生页面失效时，要从磁盘中调入一页到主存。如果主存所有页面都已经被占用，此时必须从主存储器中淘汰掉一个不常使用的页面，以便腾出主存空间来存放新调入的页面。

◆ 评价页面替换算法好坏的标准

一是命中率要高，二是算法要容易实现。



随机算法 (RAND 算法)

◆ 算法

利用软件或硬件的随机数发生器来确定主存中要被替换的页面。

◆ 特点

算法简单，容易实现；但没有利用历史信息，没有反映程序的局部性，命中率低。



先进先出算法

(FIFO算法)

◆ 算法

选择最先调入主存的页面作为要被替换的页面。

◆ 特点

比较容易实现，利用了历史信息，但没有反映程序的局部性。



近期最少使用算法

(LFU算法)

◆ 算法

选择近期最少访问的页面作为要被替换的页面。

◆ 特点

既充分利用了历史信息，又反映了程序的局部性，但实现起来非常困难。



最久没有使用算法

(LRU算法)

◆ 算法

选择近期最久没有被访问过的页面作为要被替换的页面。

◆ 特点

既充分利用了历史信息，又反映了程序的局部性，而且实现起来比较容易。



最优替换算法 (OPT算法)

◆ 算法

选择将来最久不被访问的页面作为要被替换的页面。

◆ 特点

OPT算法是一种理想化的算法，可用来作为评价其它页面替换算法好坏的标准。



例子

一个程序共有5个页面组成，程序执行过程中的页地址流如下：

P1, P2, P1, P5, P4, P1, P3, P4, P2, P4

假设分配给这个程序的主存储器共有3个页面。
给出**FIFO**、**LRU**、**OPT** 三种页面替换算法对这3页主存的使用情况，包括调入、替换和命中等。



在虚拟存储器中，实际上有可能采用只有**FIFO**和**LRU**两种算法。

时间	1	2	3	4	5	6	7	8	9	10	实际命中 次数
页地址流	P1	P2	P1	P5	P4	P1	P3	P4	P2	P4	
FIFO 算法	1	1	1	1 *	4	4	4 *	4 *	2	2	2次
		2	2	2	2 *	1	1	1	1 *	4	
				5	5	5 *	3	3	3	3 *	
	调入	调入	命中	调入	替换	替换	替换	命中	替换	替换	
LRU 算法	1	1	1	1	1 *	1	1	1 *	2	2	4次
		2	2	2 *	4	4	4 *	4	4	4	
				5	5	5 *	3	3	3 *	3 *	
	调入	调入	命中	调入	替换	命中	替换	命中	替换	命中	
OPT 算法	1	1	1	1	1	1 *	3 *	3 *	3	3	5次
		2	2	2	2 *	2	2	2	2	2	
				5 *	4	4	4	4	4	4	
	调入	调入	命中	调入	替换	命中	替换	命中	命中	命中	

三种页面替换算法对同一个页地址流的调度过程



堆栈型算法

◆ 定义

对任意一个程序的页地址流作两次主存页面数分配，分别分配 m 个主存页面和 n 个主存页面，并且有 $m \leq n$ 。如果在任何时刻 t ，主存页面数集合 B_t 都满足关系： $B_t(m) \subseteq B_t(n)$ ，则这类算法称为堆栈型替换算法。

◆ 基本思想

随着分配给程序的主存页面数增加，主存的命中率也提高，至少不下降。

◆ 例子

LFU、LRU和OPT算法是堆栈型替换算法，而FIFO算法不是。

时间 t	1	2	3	4	5	6	7	8	9	10	11	12	实际 命中次数
页地址流	P1	P2	P3	P4	P1	P2	P5	P1	P2	P3	P4	P5	
主存页面数 $n=3$	1	1	1*	4	4	4*	5	5	5	5	5	5*	3 次
		2	2	2*	1	1	1	1	1*	3	3	3	
			3	3	3*	2	2	2	2	2*	4	4	
	调入	调入	调入	替换	替换	替换	替换	命中	命中	替换	替换	命中	
主存页面数 $n=4$	1	1	1	1	1	1*	5	5	5	5*	4	4	2 次
		2	2	2	2	2	2*	1	1	1	1*	5	
			3	3	3	3	3	3*	2	2	2	2*	
				4	4	4	4	4	4*	3	3	3	
	调入	调入	调入	调入	命中	命中	替换	替换	替换	替换	替换	替换	

FIFO 替换算法是非堆栈型算法



提高主存命中率的方法

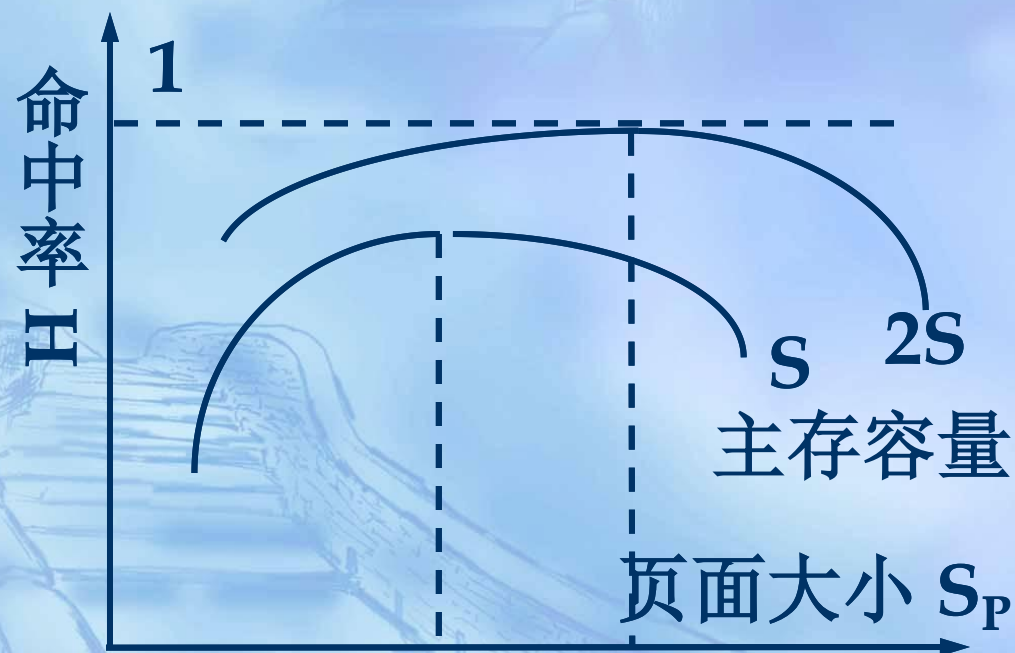
影响主存命中率的主要因素：

- ◆ 程序执行过程中的页地址流分布情况
- ◆ 所采用的页面替换算法
- ◆ 页面大小
- ◆ 主存容量
- ◆ 所采用的页面调度算法



页面大小的选择

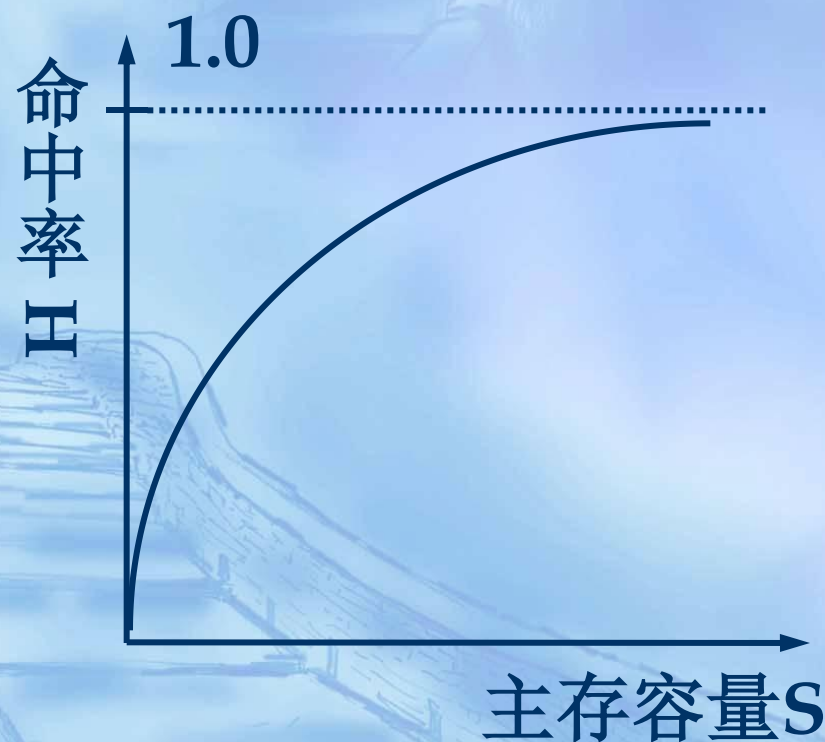
页面大小的选择一般要通过
对典型程序的模拟实验来确定，
早期一般为**1KB**，
目前大多为**4KB**、
8KB、**16KB**。





主存容量

主存命中率 H 随着分配给该程序的主存容量 S 的增加而单调上升。





页面调度算法

- ◆ **全取式**

该用和能调入的全部调入。

- ◆ **请求式**

当使用到的时候，再调入主存。

- ◆ **预取式**

在程序重新开始运行之前，把上次停止运行前一段时间内用到的页面先调入到主存储器，然后才开始运行程序。但如果调入的页面用不上，则浪费了调入的时间，占用了主存资源。



Cache存储系统

存储系统	Cache	虚拟存储器
要达到的目标	提高速度	扩大容量
实现方法	全部硬件	软件为主 硬件为辅
两级存储器速度比	3~10倍	10^5 倍
页(块)大小	1~16字	1KB~16KB
等效存储容量	主存储器	虚拟存储器
透明性	对系统和 应用程序员	仅对应用 程序员
不命中时处理方式	等待主存储器	进程切换

Cache存储系统与虚拟存储系统比较

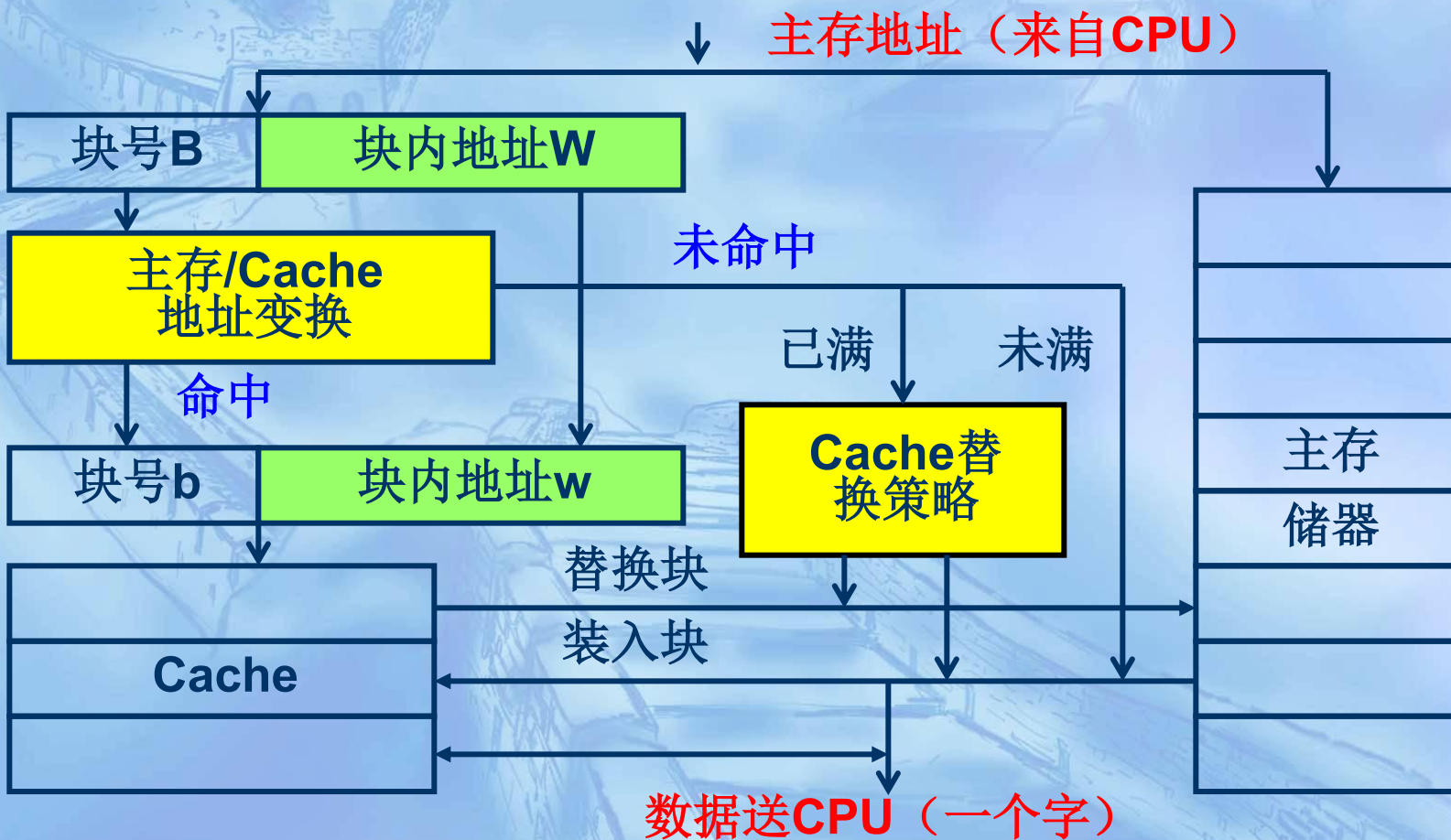


Cache存储系统

- ◆ 基本工作原理
- ◆ 地址映象与变换方法
- ◆ Cache替换算法
- ◆ Cache的一致性问题
- ◆ Cache性能



基本工作原理





地址映象与变换方法

- ◆ 基本概念
- ◆ 直接映象及其变换
- ◆ 全相联映象及其变换
- ◆ 组相联映象及其变换



基本概念

◆ 地址映象

把存放在主存中的程序按照某种规则装入到**Cache**中，并建立主存地址与**Cache**地址之间的对应关系。

◆ 地址变换

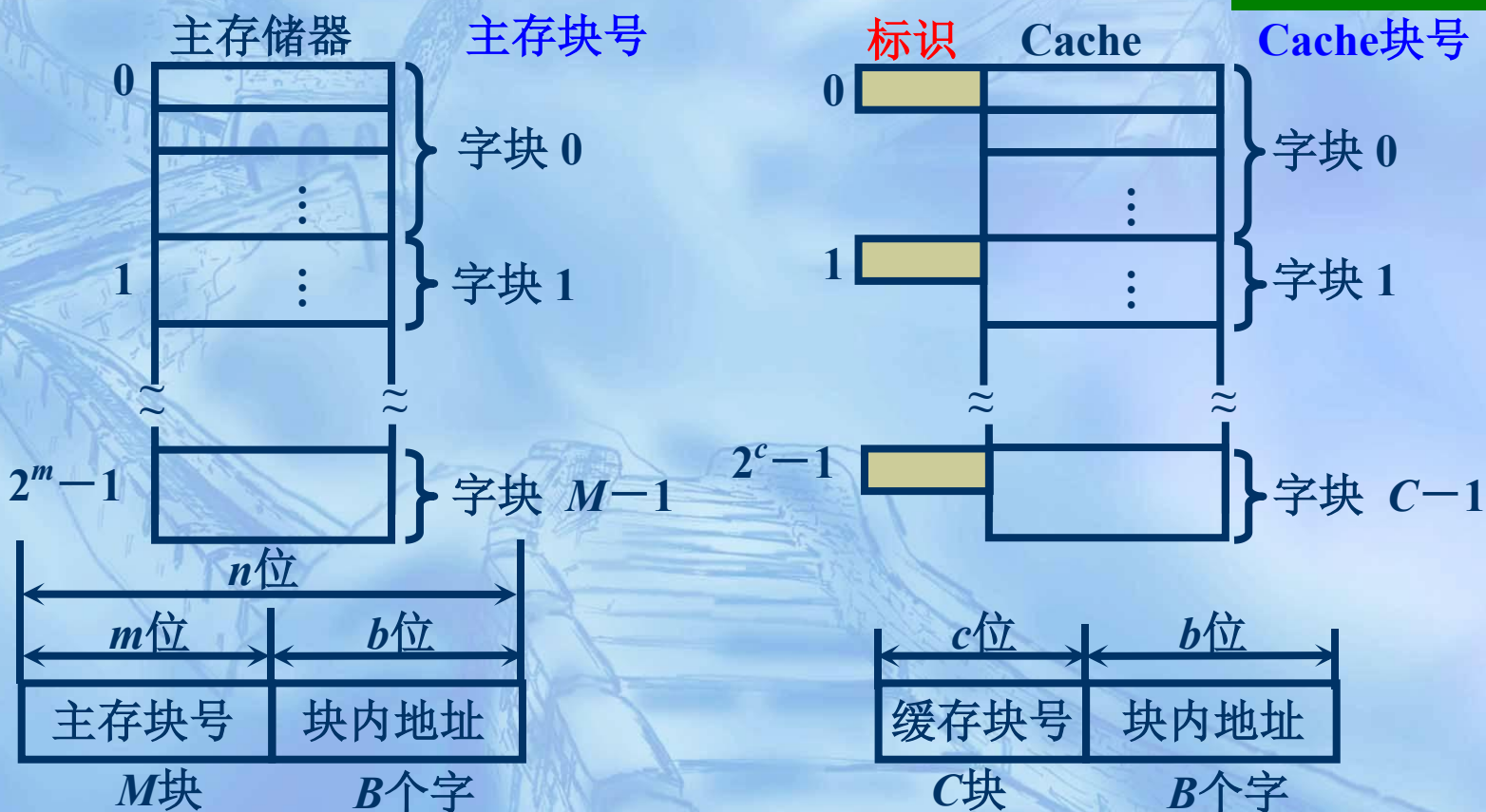
当程序已经装入到**Cache**之后，在实际运行过程中，把主存地址变换成**Cache**地址。

◆ 选择原则

地址变换的硬件要容易实现；地址变换的速度要快；**Cache**空间利用率要高；发生块冲突的概率要小。



主存和Cache的编址



按块存储，块长 B 相同，**标识**记录与某Cache块建立了对应关系的主存块号



直接映象

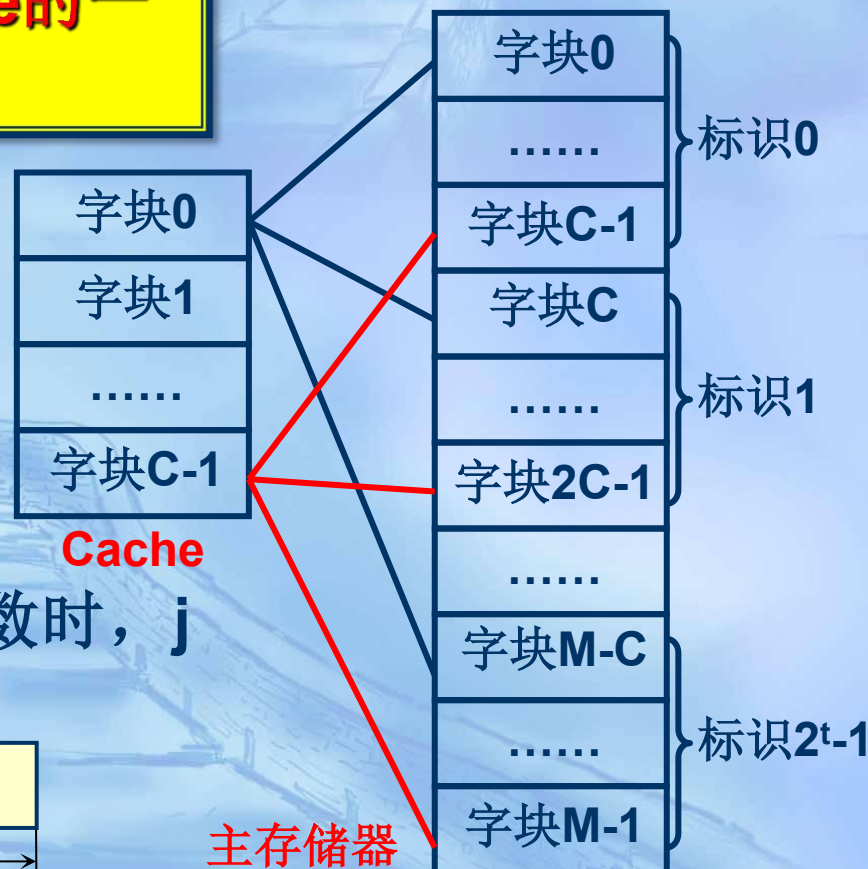
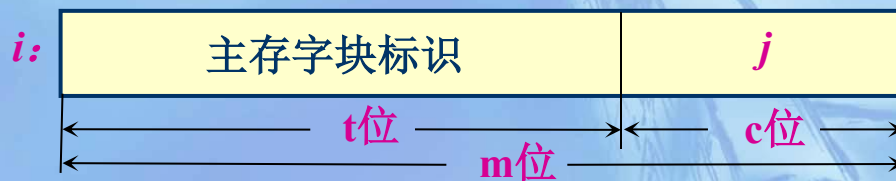
主存中一块只能映象到Cache的一个特定的块中（循环分配）。

对于主存的第*i*块，若它映象到Cache的第*j*块，则：

$$j = i \bmod C$$

（*C*为Cache的块数）

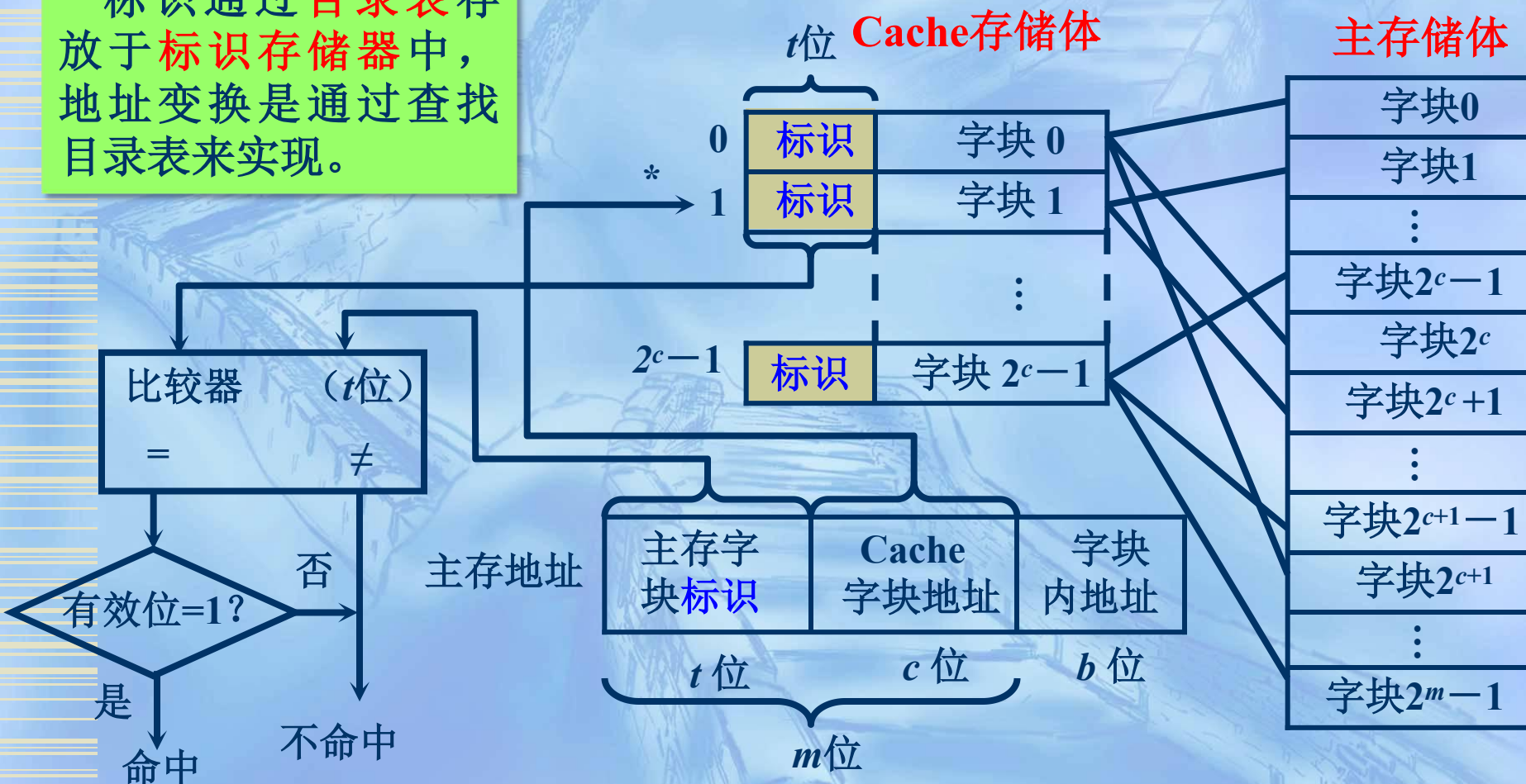
设 $C=2^c$ ，则当表示为二进制数时，*j*实际上就是*i*的低*c*位：





直接地址变换

标识通过目录表存放于标识存储器中，地址变换是通过查找目录表来实现。





特点

优点:

- ◆ 代价相对较低（硬件实现简单，无需相联存储器）
- ◆ 速度相对较快

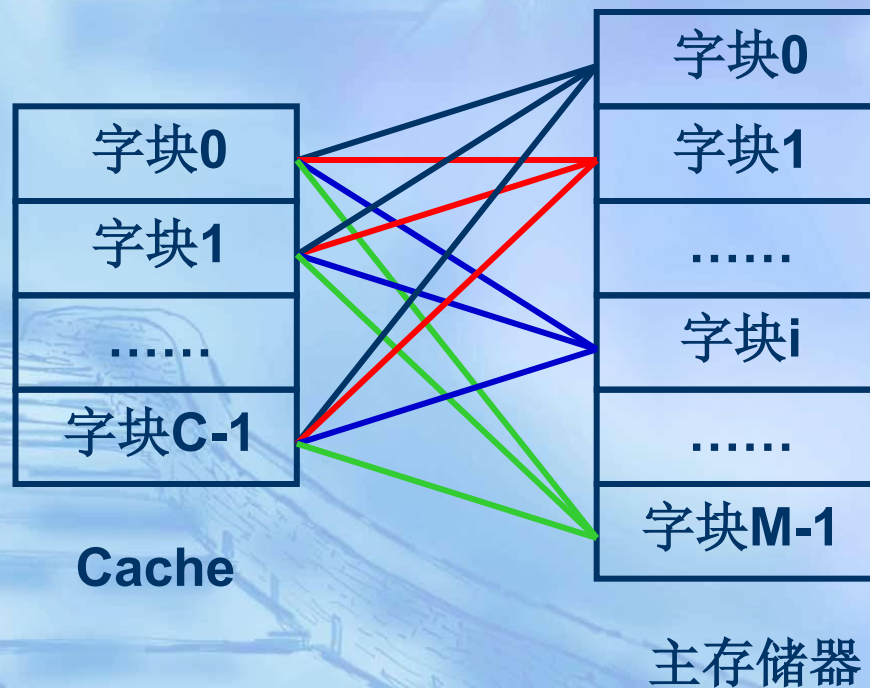
缺点:

- ◆ **Cache**的块冲突概率很高
- ◆ **Cache**的空间利用率很低



全相联映象

主存中的任意一块都可以映象到Cache中的任意一块（随便坐）。如果Cache的块数为C，主存的块数为M，映象关系共有： $C \times M$ 种。





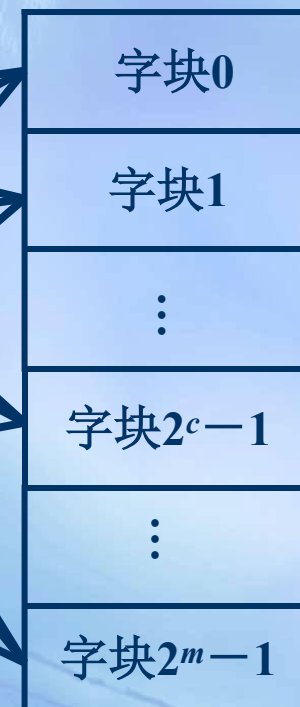
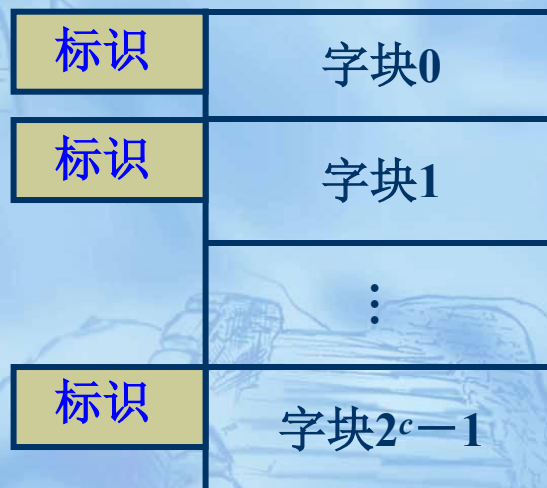
全相联地址变换

查找通常采用
“按内容寻址”
的相联存储器

Cache 存储器

主存储器

$$m = t + c$$



主存地址



$$m = t + c \text{ 位}$$

$$b \text{ 位}$$



特点

优点:

- ◆ **Cache**的块冲突概率最低;
- ◆ **Cache**的空间利用率最高。

缺点:

- ◆ 代价相对较大（相联存储器）;
- ◆ 速度相对较低（相联比较）。



组相联映象

组相联映象是目前**Cache**中用得较多的一种方法，它是介于全相联映象和直接映象之间的一种折中方案。它将主存和**Cache**按同样大小划分成块，**Cache**还划分成组，映象规则：**主存中的每一块可以被放置到Cache中唯一的一个组中（直接映象）的任何一个位置（全相联映象）。**

- 全相联映象和直接映象是组相联映象的两个极端。
- 如果一个组内有 n 块，则称为 n -路组相联。



组相联：主存中的每一块可以被放置到Cache中唯一的一个组中的任何一个位置。

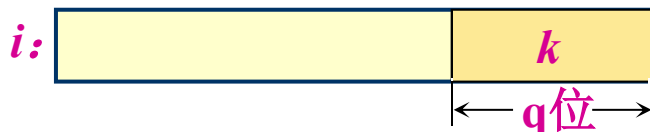
- ✓ 将Cache划分为Q组，每组有n块（n-路组相联）
- ✓ 组的选择常采用位选择算法

若主存第i块映象到第k组，则

$$k = i \bmod Q$$

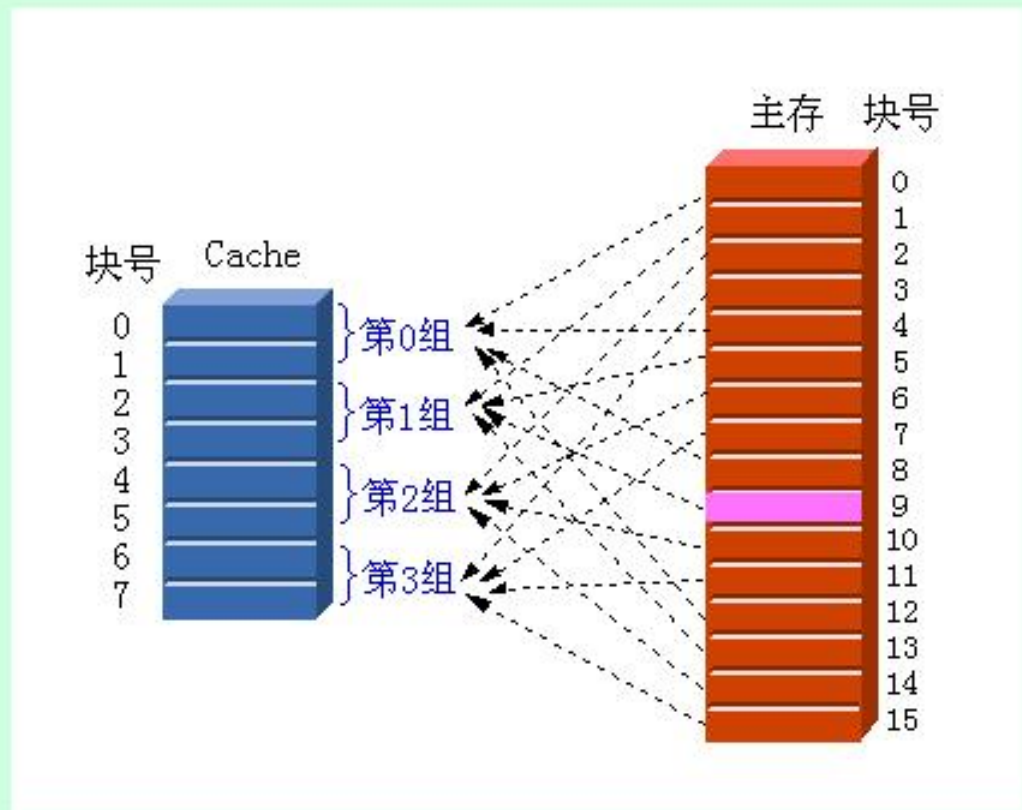
（Q为Cache的组数）

设 $Q = 2^q$ ，则当表示为二进制数时，k实际上就是i的低q位：



低q位以及直接映象中的低c位通常称为索引。

组相联映射 (举例)

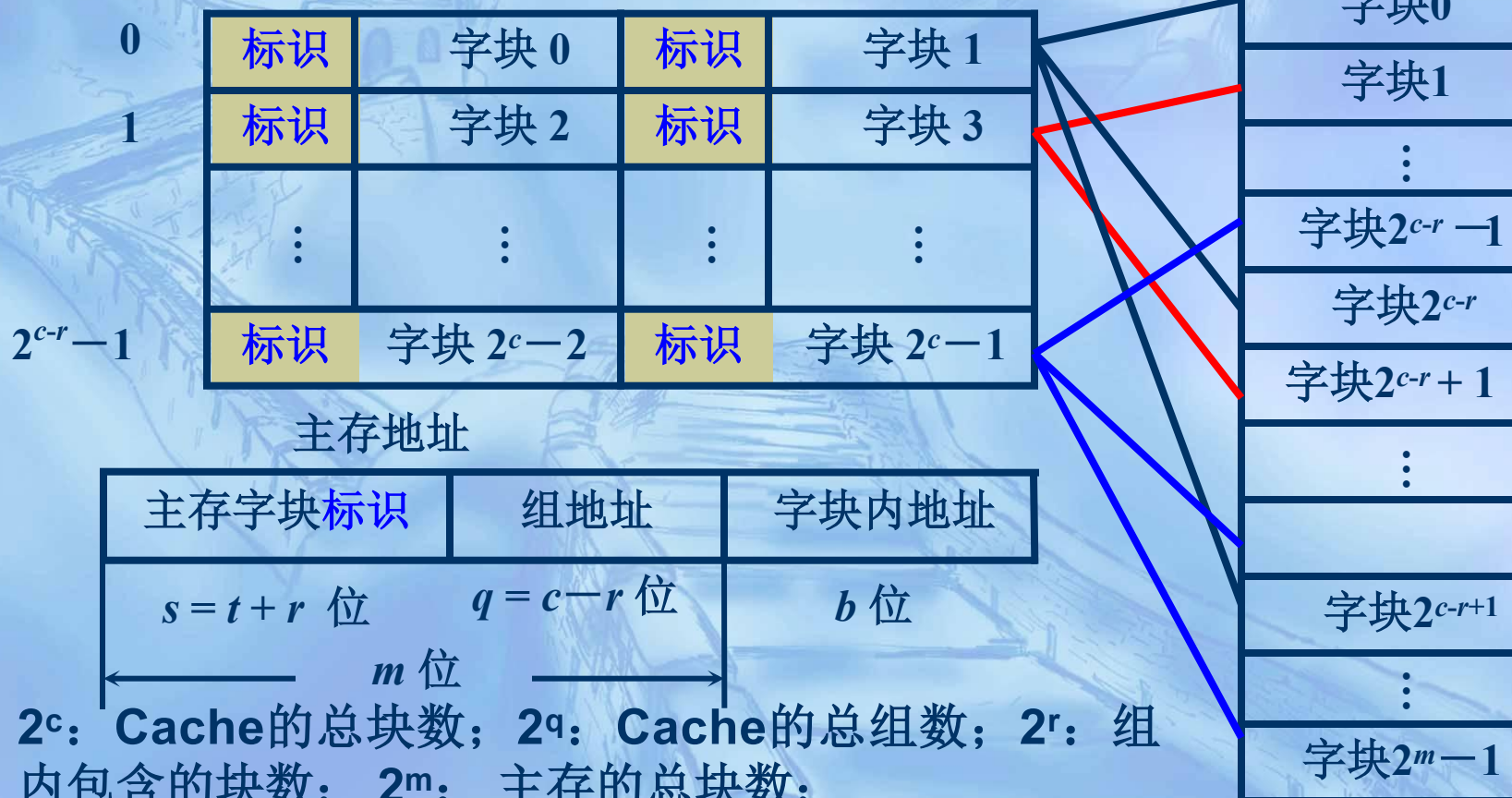




组相联地址变换

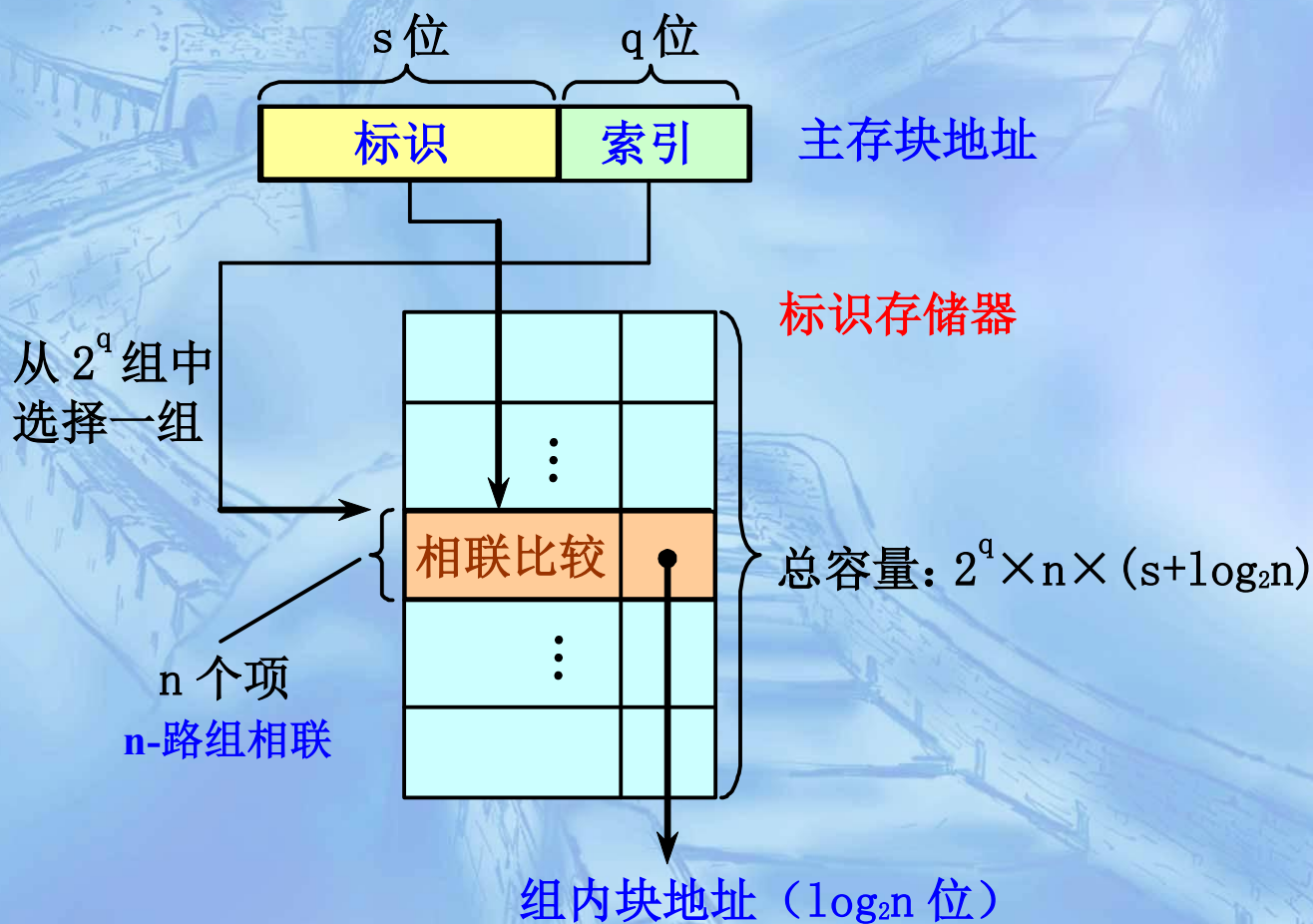
组 **Cache** 共 Q 组，每组内两块 ($r = 1$)

主存储器





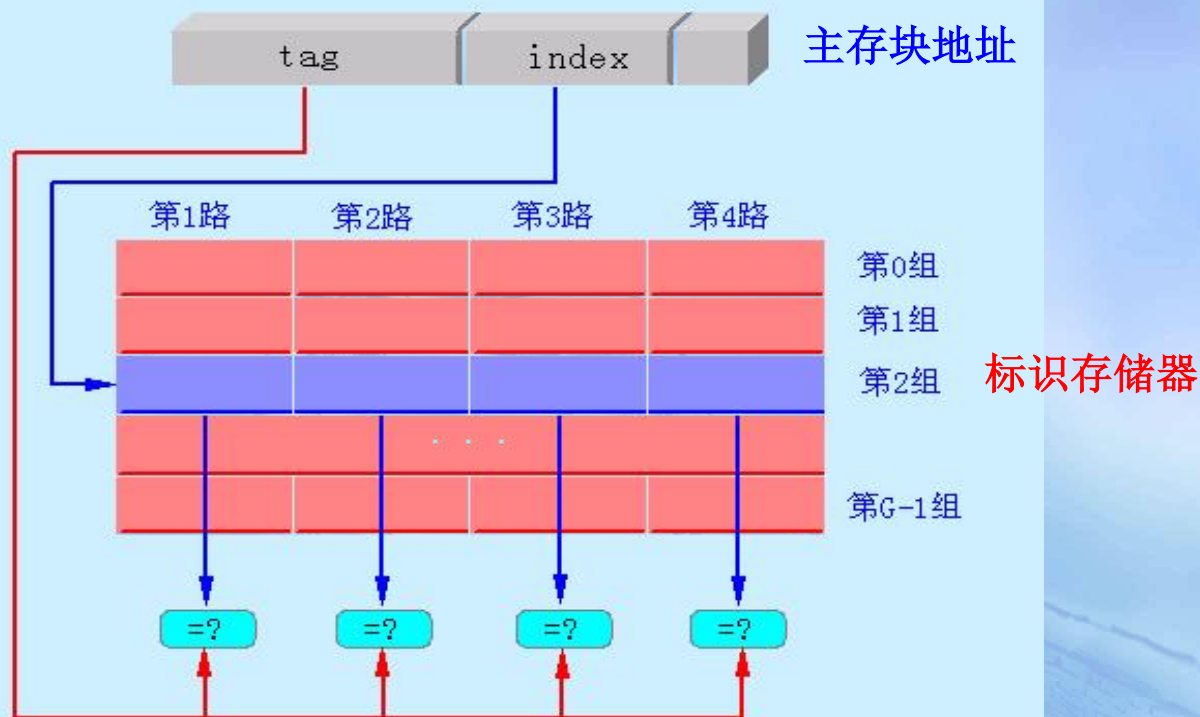
用相联存储器实现并行查找





用单体多字存储器实现并行查找

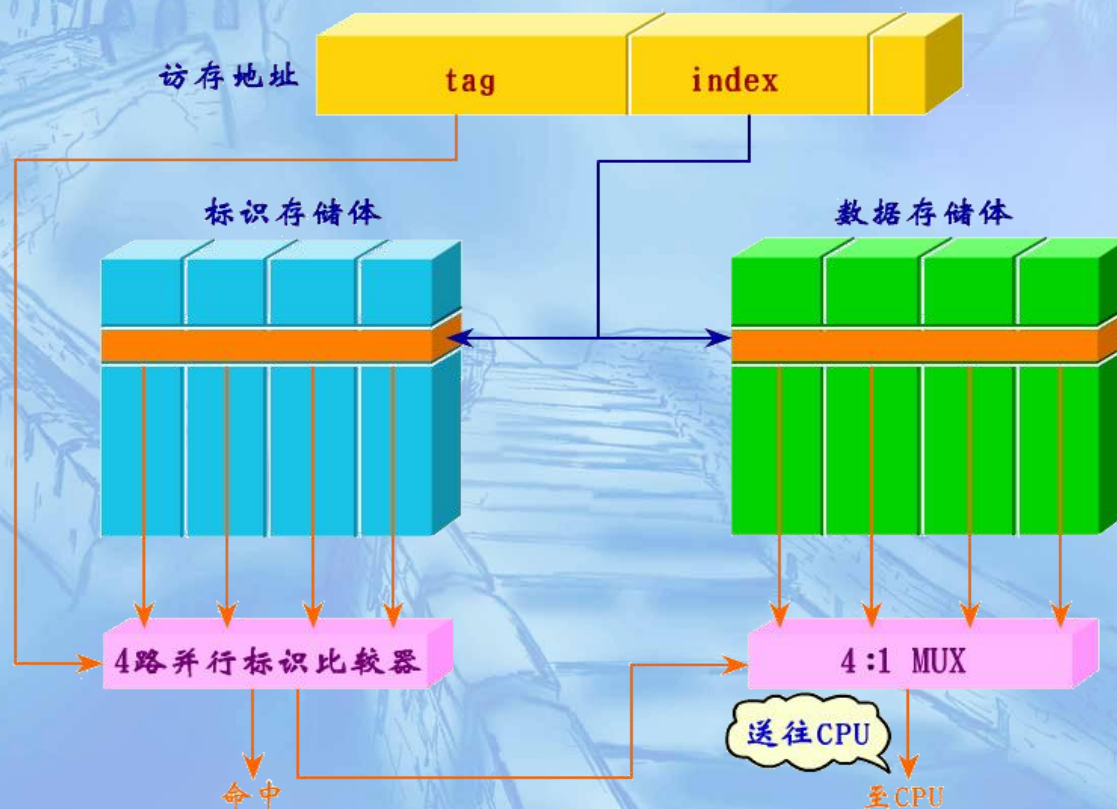
4路组相联并行标识比较





用单体多字存储器实现并行查找

4路组相联Cache的查找过程





Cache替换算法

- ◆ 基本概念
- ◆ 轮换法及其实现
- ◆ LRU算法及其实现
- ◆ 堆栈法
- ◆ 比较对法



基本概念

◆ 为什么需要Cache替换算法？

当发生块失效，且可以装入新调入块的几个Cache块都已经被装满时，此时需要Cache替换算法。直接映象方式实际上不需要替换算法，而全相联映象方式的替换算法最复杂。

◆ Cache替换算法和虚拟存储器替换算法

前者全部用硬件实现，后者主要用软件实现。



轮换法及其实现

轮换法本质上是一种**FIFO**算法，通常用于**组相联映象和地址变换方式**中，常见的有两种实现方法：

- ◆ 每块一个计数器
- ◆ 每组一个计数器



每块一个计数器

在块表中为Cache的每块都设置一个替换计数器，长度与“组内块号”字段相同。
工作方法：**装入或替换时**，被装入或替换的块计数器置**0**，同组其他块计数器加**1**；**命中时**，块计数器不变；**需替换时**，同组中计数器值最大的块被替换。



举 例

Solar-16/65机：Cache采用组相联，每组4块。

序列	初始值	装入块00	装入块01	装入块10	装入块11	替换块00
块00	00	00	01	10	11	00
块01	00	01	00	01	10	11
块10	00	01	10	00	01	10
块11	00	01	10	11	00	01



每组一个计数器

每组设置一个替换计数器，长度与“组内块号”字段相同。工作方法：本组有替换时，计数器加1，计数器的值就是要被替换出去的块号。



LRU算法及其实现

在块表中为Cache的每块都设置一个替换计数器，长度与“组内块号”字段相同。
工作方法：**装入或替换时**，被装入或替换的块计数器置**0**，同组其他块计数器加**1**；**命中时**，命中块计数器置**0**，同组中比他置**0**前的值小的计数器加**1**，其他不变；**需替换时**，同组中计数器值最大（一般为全**1**）的块被替换。



举 例

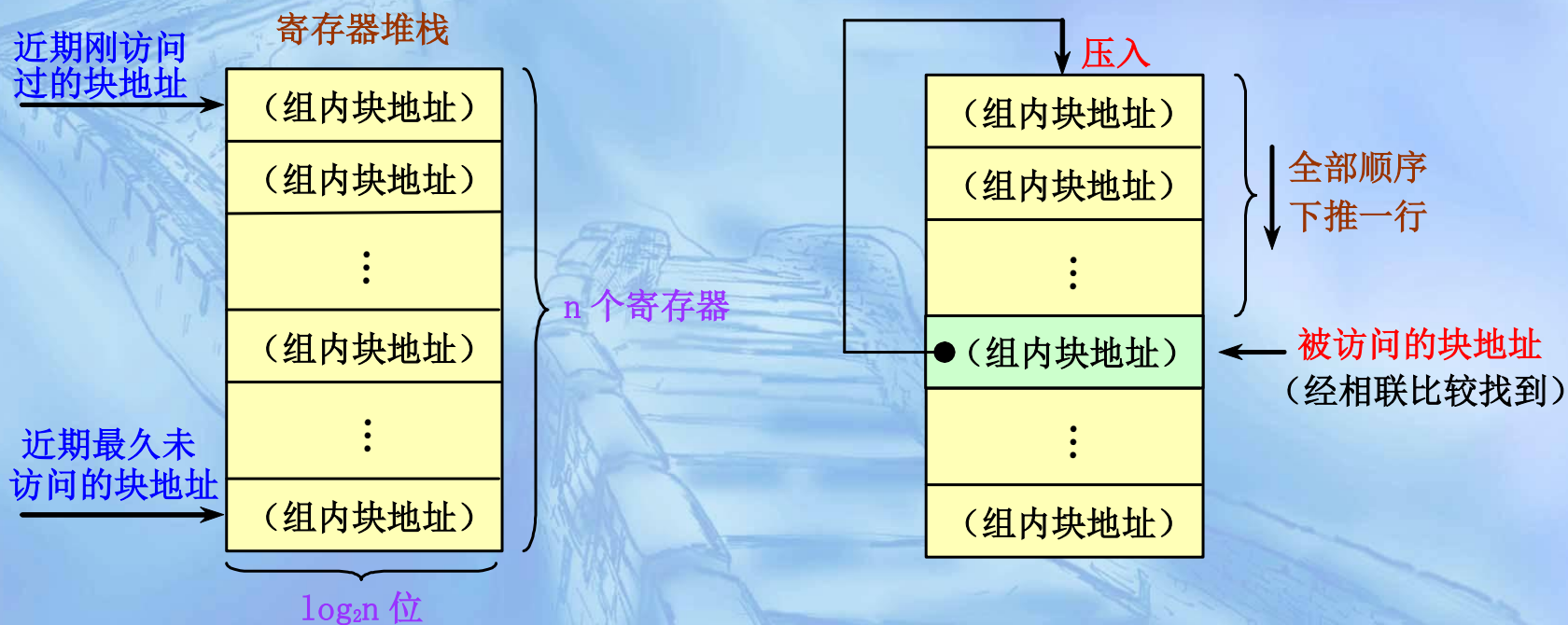
IBM 370/165机：Cache采用组相联，每组4块。

块地址流	主存块1		主存块2		主存块3		主存块4		主存块5		主存块4	
	块号	计数器	块号	计数器	块号	计数器	块号	计数器	块号	计数器	块号	计数器
Cache块0	1	00	1	01	1	10	1	11	5	00	5	01
Cache块1		01	2	00	2	01	2	10	2	11	2	11
Cache块2		01		10	3	00	3	01	3	10	3	10
Cache块3		01		10		11	4	00	4	01	4	00
动作	装入		装入		装入		装入		替换		命中	



堆栈法

堆栈法本质上是一种**LRU**算法，用栈顶到栈底的先后次序来记录**Cache**同一组内的各个块被访问的先后次序，栈顶是最近被访问过的块，栈底是最久没有被访问过的块。



(a) 用位置记录访问的先后次序

(b) 发生访问时所进行的操作



堆栈法

- ◆ 堆栈法所需要的硬件
 - 需要为每一组都设置一个项数与相联度相同的小堆栈，每一项的位数为 $\log_2 n$ 位。
- ◆ 硬件堆栈所需的功能
 - 相联比较
 - 能全部下移、部分下移和从中间取出一项的功能
- ◆ 速度较低，成本较高，只适用于相联度较小的LRU算法



比较对法

比较对法本质上是一种**LRU**算法，采用硬联逻辑实现。

一个两态的器件（触发器）能够记录两个块之间的先后顺序，多个块之间的先后顺序可以用多个两态器件的组合来实现，从而可以在多个块中找出最久没有被访问过的那个块。



举 例 —— 设计

以组中有三个块（**A**、**B**和**C**）为例：三块之间的组合共有 $C_3^2=3$ 种（**AB**、**AC**、**BC**），可以采用3个两态的触发器来表示，用 $T_{AB}=1$ 表示**B**块比**A**块更久没有被访问，其他类似。则三块最久没有被访问过的表达式为：

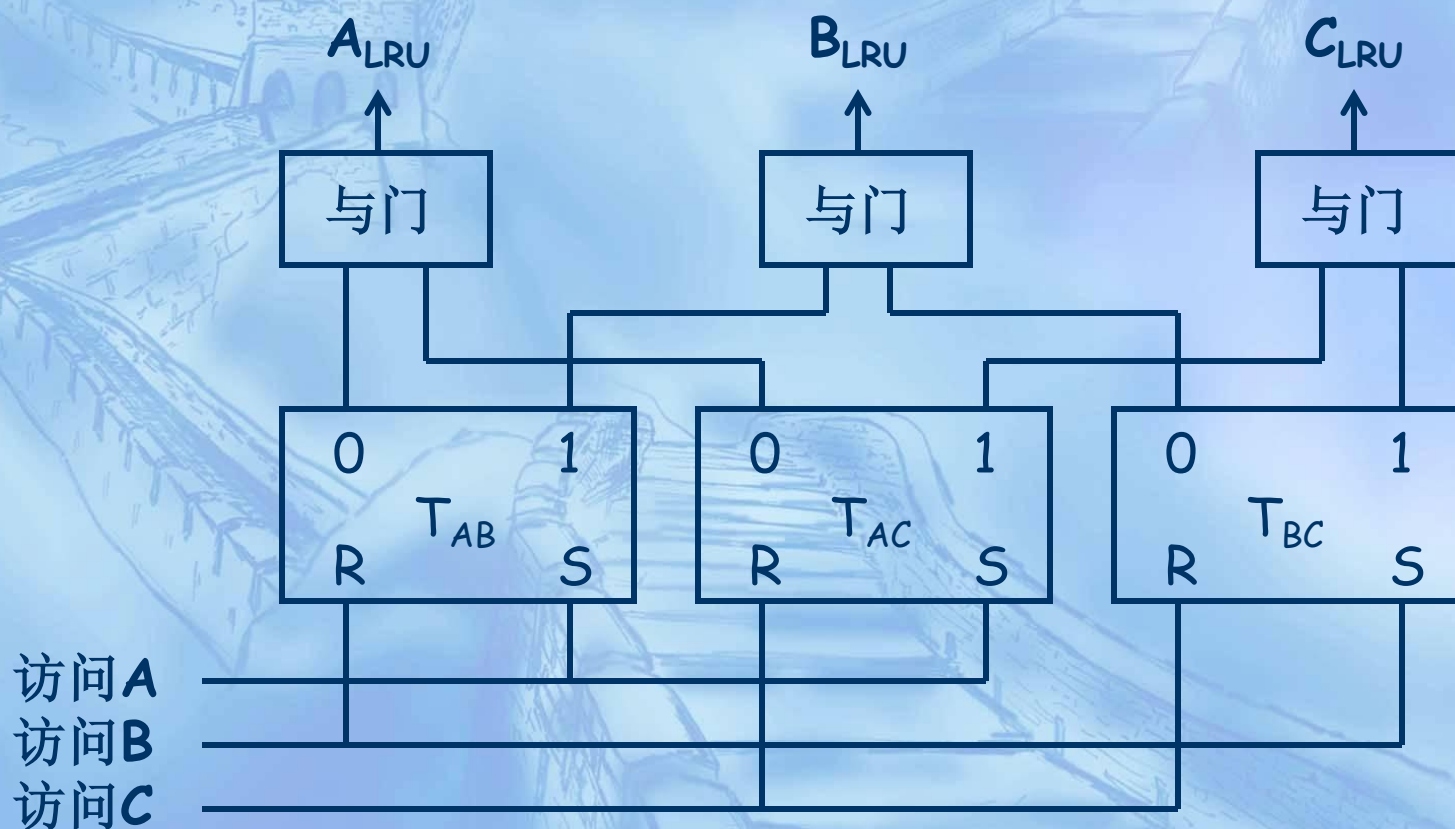
$$A_{LRU} = \overline{T_{AB}} \cdot T_{BC} \cdot \overline{T_{AC}} + \overline{T_{AB}} \cdot \overline{T_{BC}} \cdot \overline{T_{AC}} = \overline{T_{AB}} \cdot \overline{T_{AC}}$$

$$B_{LRU} = T_{AB} \cdot \overline{T_{BC}} \cdot T_{AC} + T_{AB} \cdot \overline{T_{BC}} \cdot \overline{T_{AC}} = T_{AB} \cdot \overline{T_{BC}}$$

$$C_{LRU} = T_{AB} \cdot T_{BC} \cdot T_{AC} + \overline{T_{AB}} \cdot T_{BC} \cdot T_{AC} = T_{AC} \cdot T_{BC}$$



举例 — 硬件



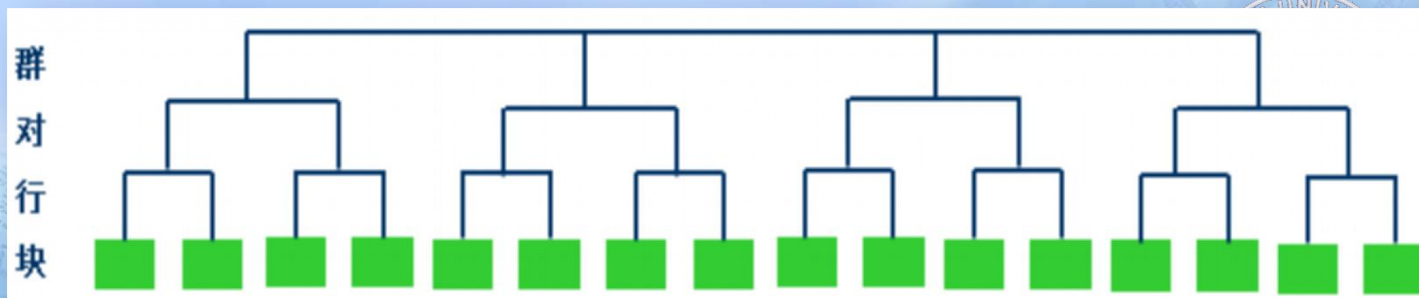


举 例 — 成本

假设每组的块数为 G_b ，则需要与门的个数为 G_b 个，每个与门的输入端个数为 $G_b - 1$ 个，需要触发器的个数为：

$$C_{G_b}^2 = \frac{G_b \cdot (G_b - 1)}{2}$$

当每组的块数较多时，所要的触发器个数和与门输入端个数很多，硬件实现的成本很高，此时可以采用**多级状态位技术**的方法来实现。



例如：在IBM 3033中，组内块数为16，可分成群、对、行3级。先分成4群，每群2对，每对2行。

◆ 计算触发器

- 选LRU群需6个触发器；
- 每群中选LRU对需要1个触发器，4个群共需要4个触发器；
- 每对中选LRU行需要1个触发器，8个对共需要8个触发器。

所需的触发器总个数为：

$$6 \text{ (选群)} + 4 \text{ (选对)} + 8 \text{ (选行)} = 18 \text{ (个)}$$

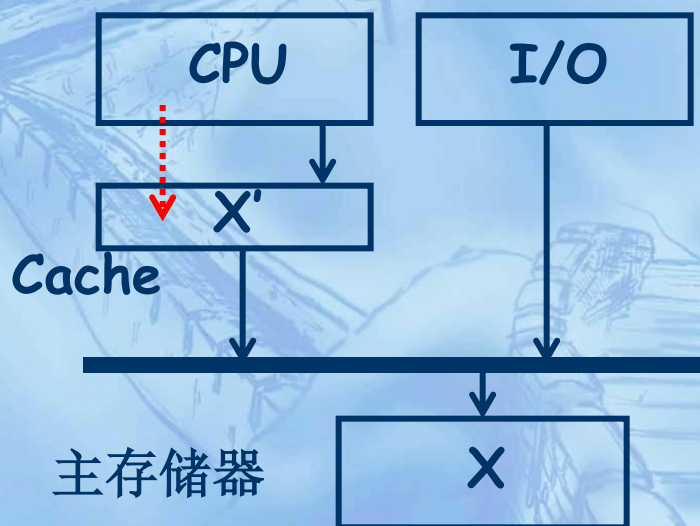
- ◆ 单级所需触发器：120个
- ◆ 提示：以牺牲速度为代价的。



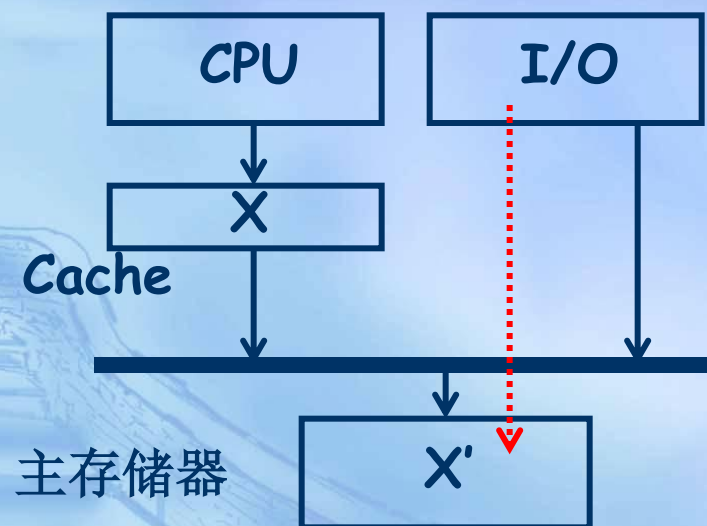
Cache的一致性问题

◆ 问题

在单处理机中会出现Cache与主存内容不一致问题。



(a) CPU写Cache



(b) I/O写主存

Cache与主存不一致的两种情况



Cache的一致性问题

◆ 解决

选择合适的Cache写策略。

- 写命中时的策略
- 写缺失时的策略



写命中时的策略

- ◆ **写直达法（写通过法： Write-Through）**
 - CPU将数据同时写入Cache和主存；
 - 为减少CPU写等待可采用写缓冲器；
 - 适用：I/O和多处理机。
- ◆ **写回法（抵触修改法： Write-Back ）**
 - CPU数据只写入Cache，不写入主存；
 - 可多次写入同一存储单元，最后只需一次写入主存；
 - 仅当替换时，才把修改过的Cache块写回到主存（设置“修改位”）；
 - 适用：多处理机。



特点比较

◆ 可靠性

写直达法要优于写回法。

◆ 与主存的通信量

一般情况下，写回法少于写直达法。

◆ 控制的复杂性

写直达法比写回法简单。

◆ 硬件实现的代价

写回法要比写直达法好。



写缺失时的策略

◆ 不按写分配法（绕写法）

在写Cache不命中时，只把所要写的字写入主存，而包括所写字在内的一个块不从主存读入Cache。

◆ 按写分配法（写时取）

在写Cache不命中时，先将包括所写字在内的一个块从主存读入Cache，然后再进行写入。



例子

问：有一个全相联映象的Cache，采用写回法，刚开始时Cache为空，有下面5个存储器操作：
Write Mem[100], Write Mem[100], Read Mem[200], Write Mem[200], Write Mem[100]; 分别求在不按写分配法和按写分配法情况下命中次数、缺失次数是多少？

答：如采用不按写分配法：命中次数为1、缺失次数为4；如采用按写分配法：命中次数为3、缺失次数为2。



提示



目前，在写回法中一般采用按写分配法，在写直达法中一般采用不按写分配法。



Cache性能

- ◆ Cache性能评价
- ◆ 提高Cache性能



Cache性能评价

- ◆ CPU执行时间
- ◆ 平均存储器访问时间 (AMAT)



CPU执行时间

CPU执行时间 = (CPU时钟周期数 + 存储器停顿周期数) × 时钟周期时间

前提:

其中:

存储器停顿

- ✓ 存储器停顿这里只考虑由Cache不命中引起的;
- ✓ Cache命中所用的时钟周期数被看作是CPU执行的时钟周期数的一部分;
- ✓ 存储器访问中将读和写操作不加区分。

$$= \text{指令数} \times \frac{\text{存储器访问次数}}{\text{指令}} \times \text{缺失率} \times \text{缺失代价}$$



例子

问：假定有一台计算机，当所有存储器访问操作都能在Cache中命中时，CPI为1.0；数据访问只有load和store指令，这些指令占全部指令的50%；缺失代价为25个时钟周期，缺失率为2%。问当所有指令都在Cache中命中时，计算机性能能提高多少？

答：Cache始终命中时的计算机性能为：

$$\begin{aligned}\text{CPU执行时间}_{\text{理想}} &= (\text{CPU时钟周期数} + \text{存储器停顿周期数}) \times \text{时钟周期} \\ &= (\text{指令数} \times \text{CPI} + 0) \times \text{时钟周期} \\ &= 1.0 \times \text{指令数} \times \text{时钟周期}\end{aligned}$$



例子 (续)

实际Cache的计算机性能为:

$$\begin{aligned}\text{CPU执行时间}_{\text{实际}} &= (\text{CPU时钟周期数} + \text{存储器停顿周期数}) \times \text{时钟周期} \\ &= (\text{指令数} \times \text{CPI} + \text{指令数} \times (1 + 0.5) \times 0.02 \times 25) \times \text{时钟周期} \\ &= 1.75 \times \text{指令数} \times \text{时钟周期}\end{aligned}$$

两者的性能比为:

$$\frac{\text{CPU执行时间}_{\text{实际}}}{\text{CPU执行时间}_{\text{理想}}} = \frac{1.75 \times \text{指令数} \times \text{时钟周期}}{1.0 \times \text{指令数} \times \text{时钟周期}} = 1.75$$

结论: 不发生Cache缺失时计算机性能是原来的1.75倍



平均存储器访问时间 (AMAT)

 T_{Cache} $T_{\text{主存}} - T_{\text{cache}}$

$$\begin{aligned} \text{AMAT} &= H \cdot T_{\text{Cache}} + (1 - H) \cdot T_{\text{主存}} \\ &= \text{命中时间} + \text{缺失率} \cdot \text{缺失代价} \end{aligned}$$



例子

问：一个由8KB的I-Cache和8KB的D-Cache所构成的分立Cache（哈佛结构）与一个16KB的统一Cache哪一个具有更低的缺失率？假设命中所需的开销为1个时钟周期，不命中的开销为50个时钟周期，统一Cache的load或store命中需花费1个时钟周期的额外开销。75%的存储器存取是指令访问。

Cache大小	I-Cache缺失率	D-Cache缺失率	统一Cache缺失率
4KB	1.78%	15.94%	7.24%
8KB	1.10%	10.19%	4.57%
16KB	0.64%	6.47%	2.87%
32KB	0.39%	4.82%	1.99%



例子 (续)

答：分立Cache的整体缺失率为：

$$(75\% \times 1.10\%) + (25\% \times 10.19\%) = 3.37\%$$

由表中可知，**16KB**的统一Cache的缺失率为**2.87%**。
因此，统一Cache结构具有较低的缺失率。

$$AMAT_{\text{split}} = 75\% \times (1 + 1.10\% \times 50) + 25\% \times (1 + 10.19\% \times 50) = 2.686$$

$$AMAT_{\text{unified}} = 75\% \times (1 + 2.87\% \times 50) + 25\% \times (1 + 1 + 2.87\% \times 50) = 2.685$$

尽管分立Cache具有较高的缺失率，但其AMAT与统一Cache的AMAT是基本相同的，可见哈佛结构有优势。
大多数现代处理器都采用分立Cache技术。



提高Cache性能

T_{Cache}

$T_{\text{主存}} - T_{\text{cache}}$

$$\begin{aligned} \text{AMAT} &= H \cdot T_{\text{Cache}} + (1 - H) \cdot T_{\text{主存}} \\ &= \text{命中时间} + \text{缺失率} \cdot \text{缺失代价} \end{aligned}$$

可见主要途径有：

- ◆ 降低缺失代价
- ◆ 降低缺失率
- ◆ 通过并行性降低缺失代价/缺失率
- ◆ 降低Cache命中时间



降低缺失代价

- ◆ 多级Cache
- ◆ 请求字处理技术
- ◆ 让读缺失优先于写
- ◆ 合并写缓冲区
- ◆ 牺牲者Cache



多级Cache

- ◆ 基本思想
- ◆ 性能分析
- ◆ 设计考虑



基本思想

通过在原始**Cache**和存储器之间增加另一级**Cache**，第一级**Cache**可以小到足以跟上飞快的**CPU**，而第二级**Cache**能够大到足以捕捉到对主存进行的大多数访问，因而可以有效降低缺失代价。



性能分析

$$\begin{aligned} \text{AMAT} &= \text{命中时间}_{L1} + \text{缺失率}_{L1} \times \text{缺失代价}_{L1} \\ &= \text{命中时间}_{L1} + \text{缺失率}_{L1} \times (\text{命中时间}_{L2} + \text{缺失率}_{L2} \times \text{缺失代价}_{L2}) \end{aligned}$$

◆ 局部缺失率

本级Cache的缺失数除以对本级Cache的存储器访问总数。例如：第一级Cache的局部缺失率为缺失率 L_1 ，第二级Cache的局部缺失率为缺失率 L_2

◆ 全局缺失率

本级Cache的缺失数除以CPU产生的存储器访问总数。例如：第一级Cache的全局缺失率为缺失率 L_1 ，第二级Cache的全局缺失率为缺失率 $L_1 \times$ 缺失率 L_2 。



例子

例：在某两级Cache中：

- (1) 假设在**1000**次访存中，**L1级Cache**失效**40**次，**L2级Cache**失效**20**次。试问：各级的局部缺失率和全局缺失率各是多少？
- (2) 假设**L2**的命中时间是**10**个时钟周期，**L2**的缺失代价是**100**时钟周期；**L1**的命中时间是**1**个时钟周期，平均每条指令访存**1.5**次，不考虑写操作的影响。问：平均访存时间是多少？每条指令的平均停顿时间是多少个时钟周期？

答：(1)

第一级Cache的缺失率（全局和局部）是： $40/1000=4\%$

第二级Cache的局部缺失率是： $20/40=50\%$

第二级Cache的全局缺失率是： $20/1000=2\%$



例子

前提:

- ✓ 存储器访问中将读和写操作不加区分;
- ✓ 第1级Cache采用写回法。

答: (2)

$$\begin{aligned} \text{AMAT} &= \text{命中时间}_{L_1} + \text{缺失率}_{L_1} \times (\text{命中时间}_{L_2} + \text{缺失率}_{L_2} \times \text{缺失代价}_{L_2}) \\ &= 1 + 4\% \times (10 + 50\% \times 100) \\ &= 3.4 \text{ 个时钟周期} \end{aligned}$$

$$\begin{aligned} \text{每次访存的平均停顿时间} &= \text{全局缺失率}_{L_1} \times \text{命中时间}_{L_2} + \\ &\quad \text{全局缺失率}_{L_2} \times \text{缺失代价}_{L_2} \\ &= 4\% \times 10 + 2\% \times 100 \\ &= 2.4 \text{ 个时钟周期} \end{aligned}$$

由于平均每条指令访存1.5次, 所以:

$$\text{每条指令的平均访存停顿时间} = 2.4 \times 1.5 = 3.6 \text{ 个时钟周期}$$



第二级Cache

- ◆ 在第二级**Cache**比第一级 **Cache**大得多的情况下，两级**Cache**的全局缺失率和容量与第二级**Cache**相同的单级**Cache**的缺失率非常接近。故可以采用单级**Cache**的知识和结论来分析。
- ◆ **局部缺失率**不是衡量第二级**Cache**的一个好指标，因为它会依赖于第一级**Cache**，且不能全面反映两级**Cache**系统的性能。故在评价第二级**Cache**时，应用**全局缺失率**这个指标。
- ◆ 第二级**Cache**不会影响**CPU**的时钟频率，只影响第一级**Cache**的缺失代价，因此其设计有更大的考虑空间。



设计考虑

◆ 第二级Cache的设计

需考虑的问题有二：

- 它能否降低CPI中的平均访存时间部分？
- 它的成本是多少？

◆ 第二级Cache的参数

- 容量

采用大容量设计，如**512 KB**，**1024 KB**。因为第一级Cache中的所有信息都可能会出现在第二级Cache中，所以第二级Cache应该比第一级Cache大得多。



设计考虑

◆ 第二级Cache的参数

➤ 相联度

第二级Cache可采用较高的相联度或伪相联方法。

➤ 块大小

为减少平均访存时间，可以让容量较小的第一级Cache采用较小的块，而让容量较大的第二级Cache采用较大的块。如 **64B**、**128B**、**256B**。

➤ 多级包容性

第一级Cache中的数据是否总是同时存在于第二级Cache中。



第二级Cache的相联度设计考虑

问：给出有关第二级Cache的以下数据：

- (1) 对于直接映象，命中时间 $L_2 = 10$ 个时钟周期
- (2) 2路组相联使命中时间增加 $10\% \times$ 时钟周期
- (3) 对于直接映象，局部缺失率 $L_2 = 25\%$
- (4) 对于2路组相联，局部缺失率 $L_2 = 20\%$
- (5) 缺失代价 $L_2 = 50$ 个时钟周期

试问第二级Cache的相联度对缺失代价的影响如何？



第二级Cache的相联度设计考虑

答： 对一个直接映象的第二级Cache来说：

缺失代价_{直接映象}, $L_1 = 10 + 25\% \times 50 = 22.5$ 个时钟周期

对于2路组相联第二级Cache来说：

缺失代价_{2路组相联}, $L_1 = 10.1 + 20\% \times 50 = 20.1$ 个时钟周期

如将第二级Cache的命中时间取整，得10/11，则：

缺失代价_{2路组相联}, $L_1 = 10 + 20\% \times 50 = 20.0$ 个时钟周期

缺失代价_{2路组相联}, $L_1 = 11 + 20\% \times 50 = 21.0$ 个时钟周期

结论： 对于第二级Cache来说，2路组相联优于直接映象



请求字处理技术

◆ 思想

因为**CPU**在同一时刻只需要块中的一个字（**请求字**），所以本技术不必等到全部块装入就可以将所需字送出，然后重新启动**CPU**。

◆ 方法

➤ 尽早重启动

按正常次序获取字，只要被请求的字一到达就将它发送到**CPU**中，让**CPU**继续执行。

➤ 请求字优先

首先向存储器请求缺失的字，一旦它到了就将它发送到**CPU**中；让**CPU**继续执行，同时装入块中的其他字。



局限性

本技术的收益取决于块的大小（块越大，收益越大）和对块中未装入部分的访问可能性。所以这种技术在以下情况下效果不大：

- **Cache块较小**
- 下一条指令正好访问同一**Cache块**的另一部分



让读缺失优先于写

◆ 问题

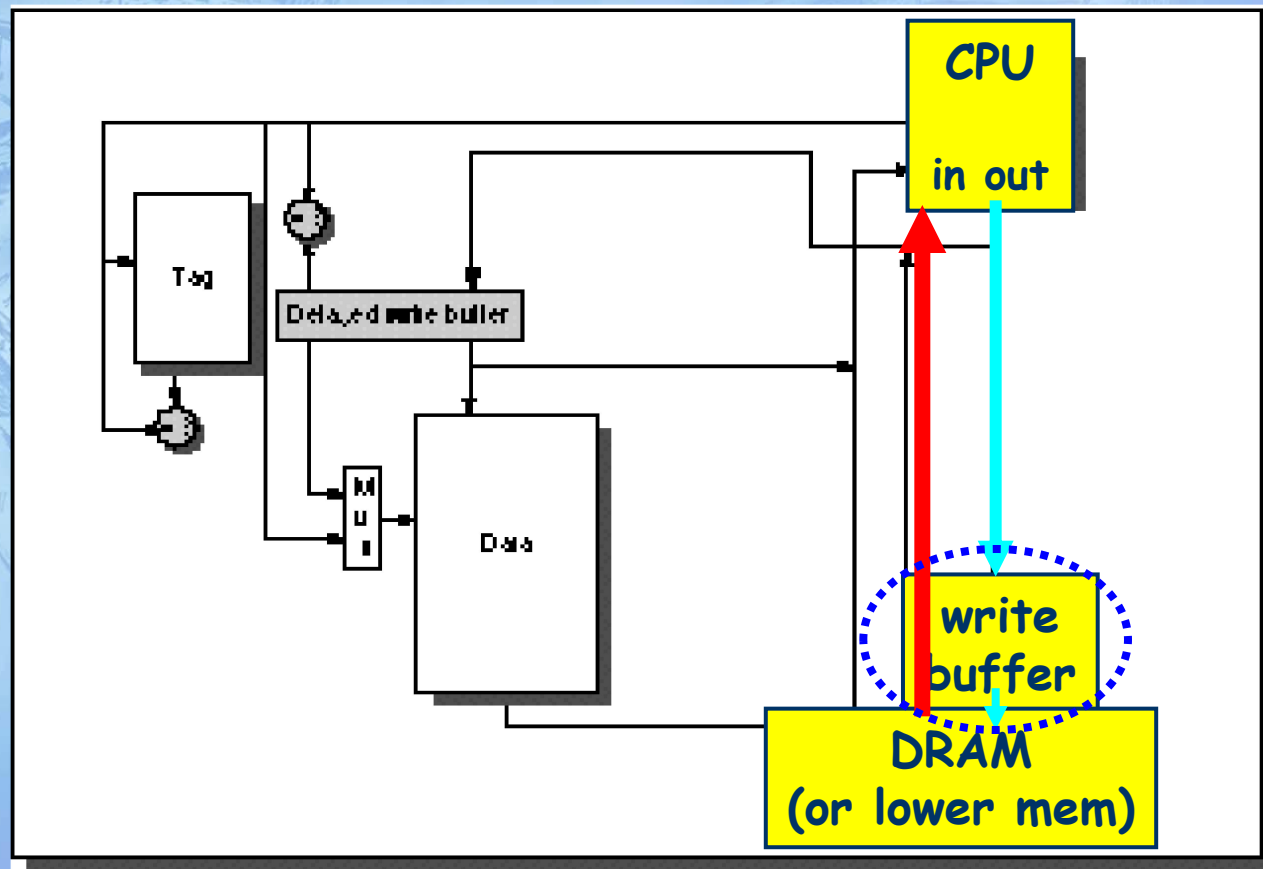
对于一个写直达的Cache，需要设置容量适中的写缓冲区（见后图）。然而写缓冲区（滞后）使得存储器访问变的复杂，因为其中可能包含读缺失时所需要的更新数据。

SW R3,512(R0) ;M[512]←R3 (Cache Index 0)
LW R1,1024(R0) ;R1 ←M[1024] (Cache Index 0)
LW R2,512(R0) ;R2 ←M[512] (Cache Index 0)





写缓冲区





让读缺失优先于写

◆ 解决

最简单的解决方法：读缺失等待，直到写缓冲区为空为止；但该方法会增加读缺失代价。另一种解决方法（**常用**）：在读缺失时查看写缓冲区中的内容，如果没有冲突（即没有地址相同）而且存储器系统可以访问，就可继续处理读缺失。



在写回法的**Cache**中，在替换块时也要使用一个简单的写缓冲，同样处理。



合并写缓冲区

- ◆ **写直达Cache:** 依靠**写缓冲**来减少对下一级存储器写操作的时间。
 - 如果写缓冲器为空，就把数据和相应地址写入该缓冲器。从**CPU**的角度来看，该写操作就算完成了。
 - 如果写缓冲器中已经有了待写入的数据，就要把这次的写入地址与写缓冲器中已有的所有地址进行比较，看是否有匹配的项。如果有地址匹配而对应的位置又是空闲的，就把这次要写入的数据与该项合并。这就叫**合并写缓冲**。
 - 如果写缓冲器满且又没有能进行写合并的项，就必须等待。



合并写缓冲区

◆ 特点

在写缓冲区中，将多个连续的数据组合起来，提高写缓冲区的空间利用率，减少因写缓冲区满而等待的时间。

Write address	V	64bit	V	64bit	V	64bit	V	64bit
100	1	Mem[100]	0		0		0	
108	1	Mem[108]	0		0		0	
116	1	Mem[116]	0		0		0	
124	1	Mem[124]	0		0		0	

Write address	V		V		V		V	
100	1	Mem[100]	1	Mem[108]	1	Mem[116]	1	Mem[124]
	0		0		0		0	
	0		0		0		0	
	0		0		0		0	



牺牲者Cache

◆ 思想

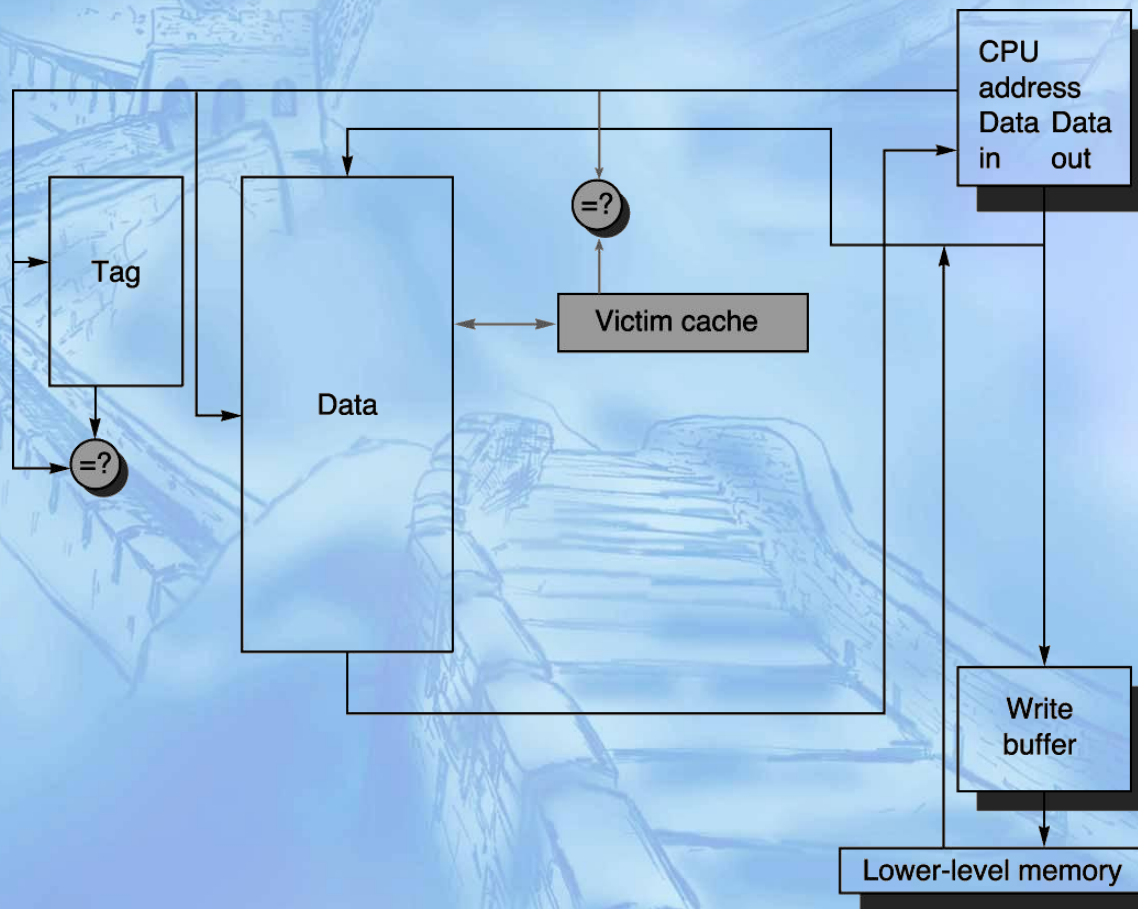
在Cache和它的替换路径之间增加一个小的、全相联的Cache（牺牲者Cache），这个牺牲者Cache中只包含Cache中因为冲突而被替换出的块（牺牲者），然后在缺失发生时，在要访问下层存储器之前，先检查牺牲者Cache，看其中是否包含有期望的数据，如果有，则牺牲块与Cache块互换（见后图）。

◆ 性能

依赖于特定的程序，一个包含4个存储字的牺牲者Cache能减少20%~90%的冲突缺失。



牺牲者Cache





降低缺失率

- ◆ 导致缺失的原因
- ◆ 降低缺失率的技术
 - 增加Cache块大小
 - 增加Cache容量
 - 增加相联度
 - 路预测和伪相联Cache
 - 编译优化



导致缺失的原因

◆ 强制（Compulsory）缺失

对一个块的第一次访问一定不在Cache中，所以该块必须被调入到Cache中（这也称为：冷启动缺失、首次访问缺失等）。

◆ 容量（Capacity）缺失

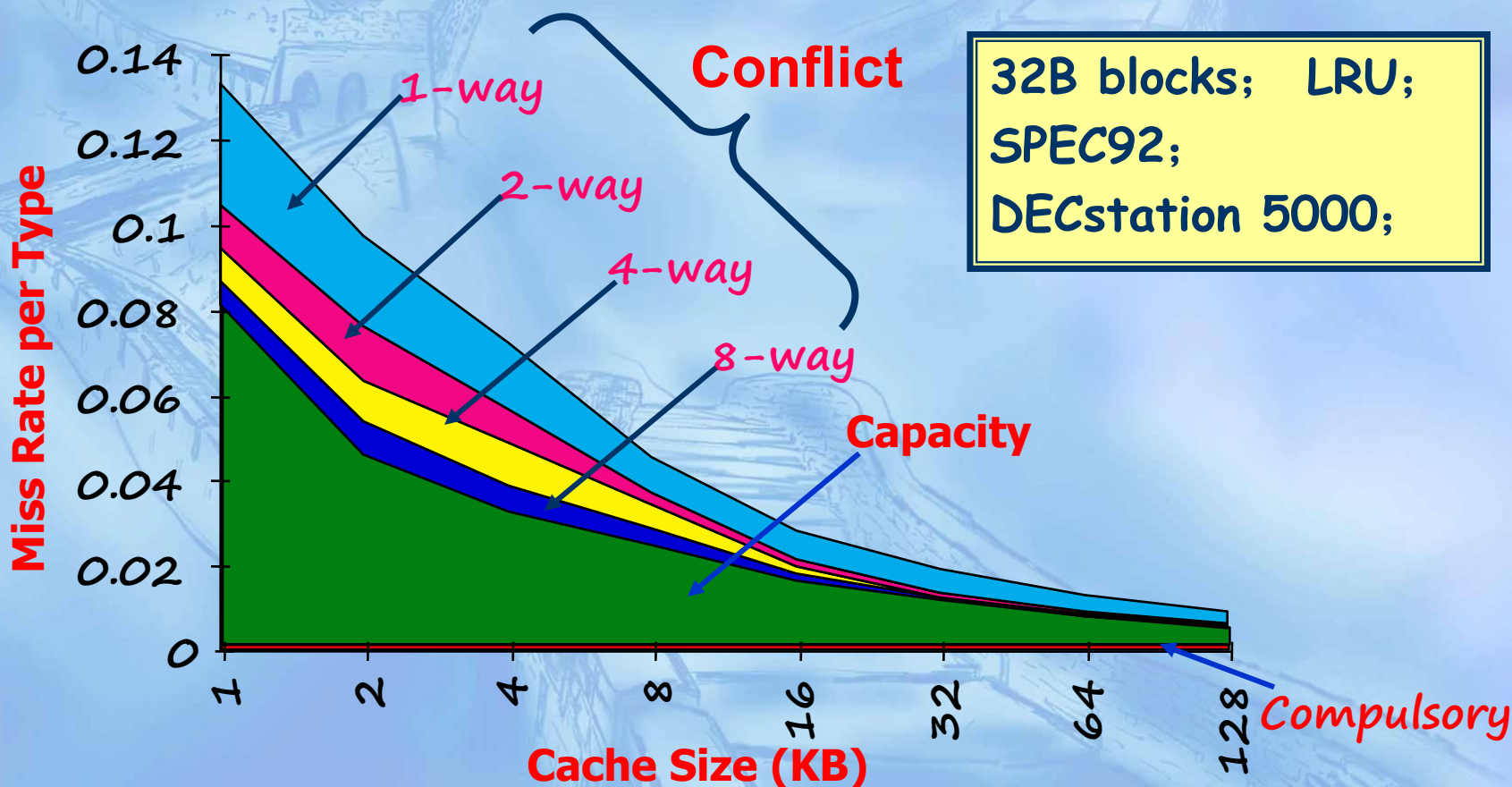
如果Cache容纳不了一个程序持续执行所需要的所有块，当某些块被替换后，若又重新被访问就会发生缺失。

◆ 冲突（Conflict）缺失

如果采用组相联/直接映象，若太多的块映射到同一组（块）中，则会出现该组中某块被别的块替换后又被重新访问，这就发生冲突缺失。

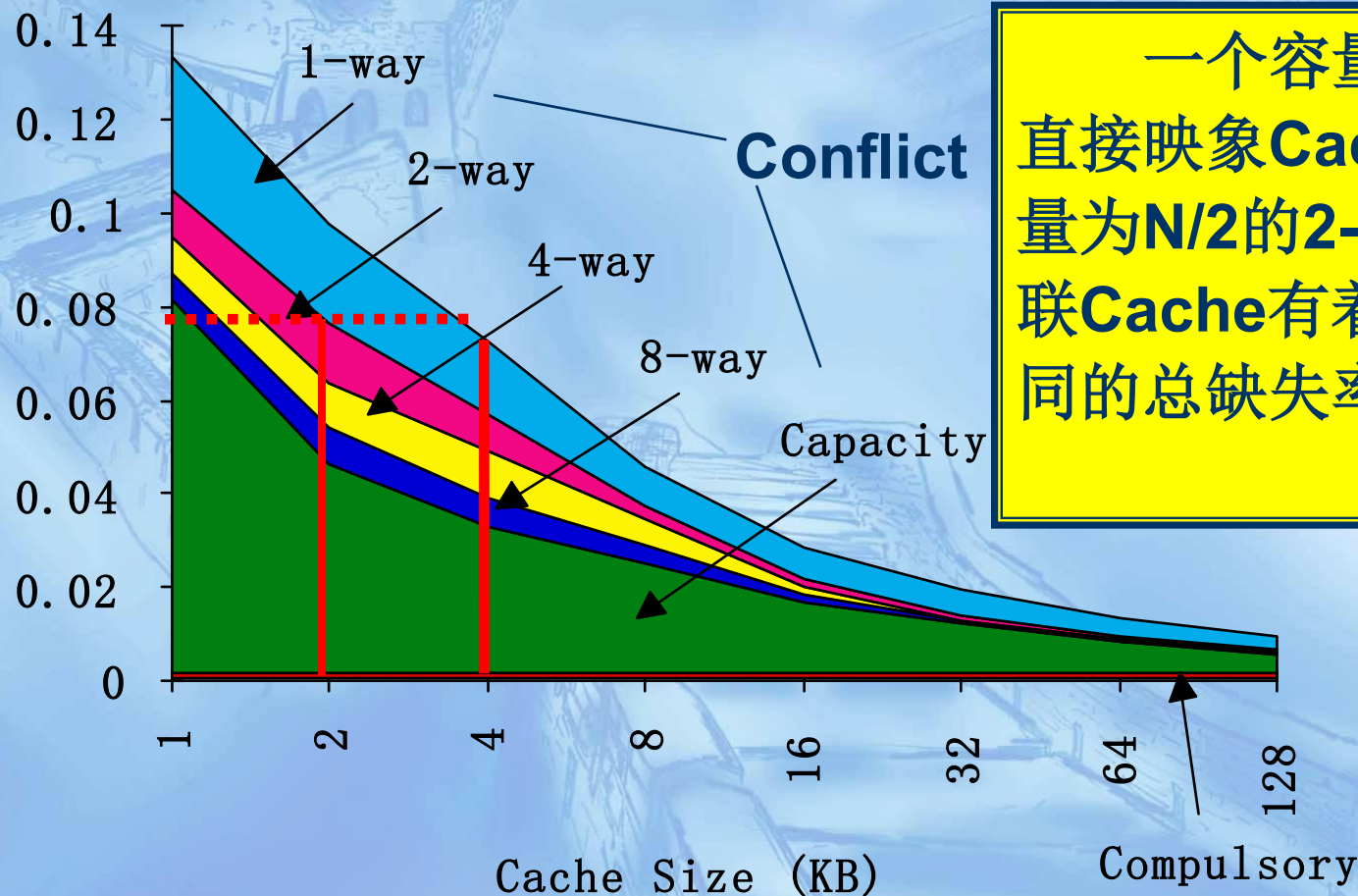


3Cs总缺失率





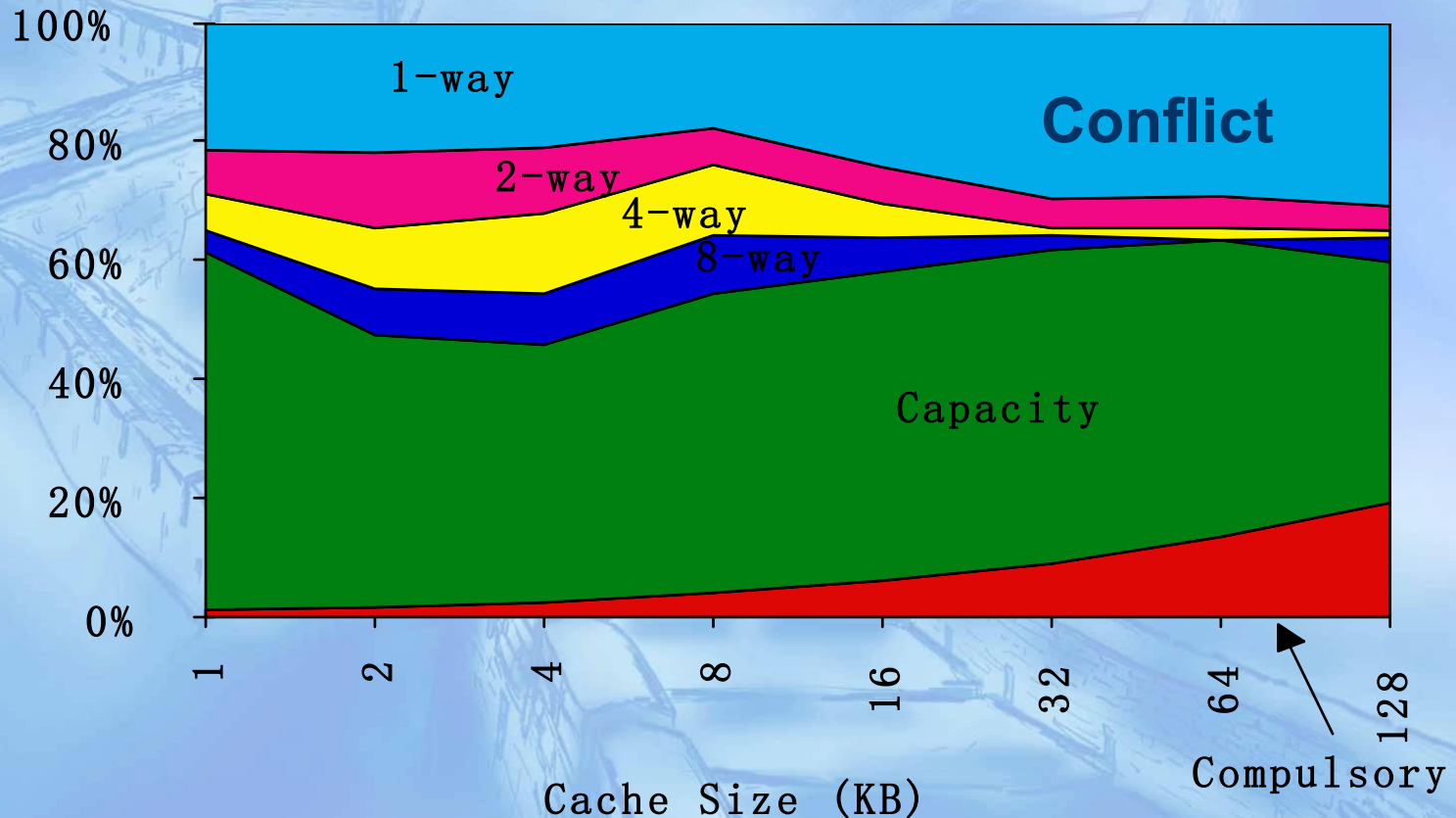
2:1 Cache经验规律



一个容量为N的
直接映象Cache同容
量为N/2的2-路组相
联Cache有着大致相
同的总缺失率。



3Cs缺失率分布 (相对值)





总结

◆ 规律

- 相联度越高，冲突缺失就越少
- 强制缺失和容量缺失不受相联度的影响
- 强制缺失不受**Cache**容量的影响，但容量缺失却随着容量的增加而减少

◆ 减少三种缺失的方法

- 强制缺失（本身很少）：增加块大小，预取
 - 容量缺失：增加容量
 - 冲突缺失：提高相联度（理想情况：全相联）
- ◆ 许多降低缺失率的方法会增加命中时间或缺失代价



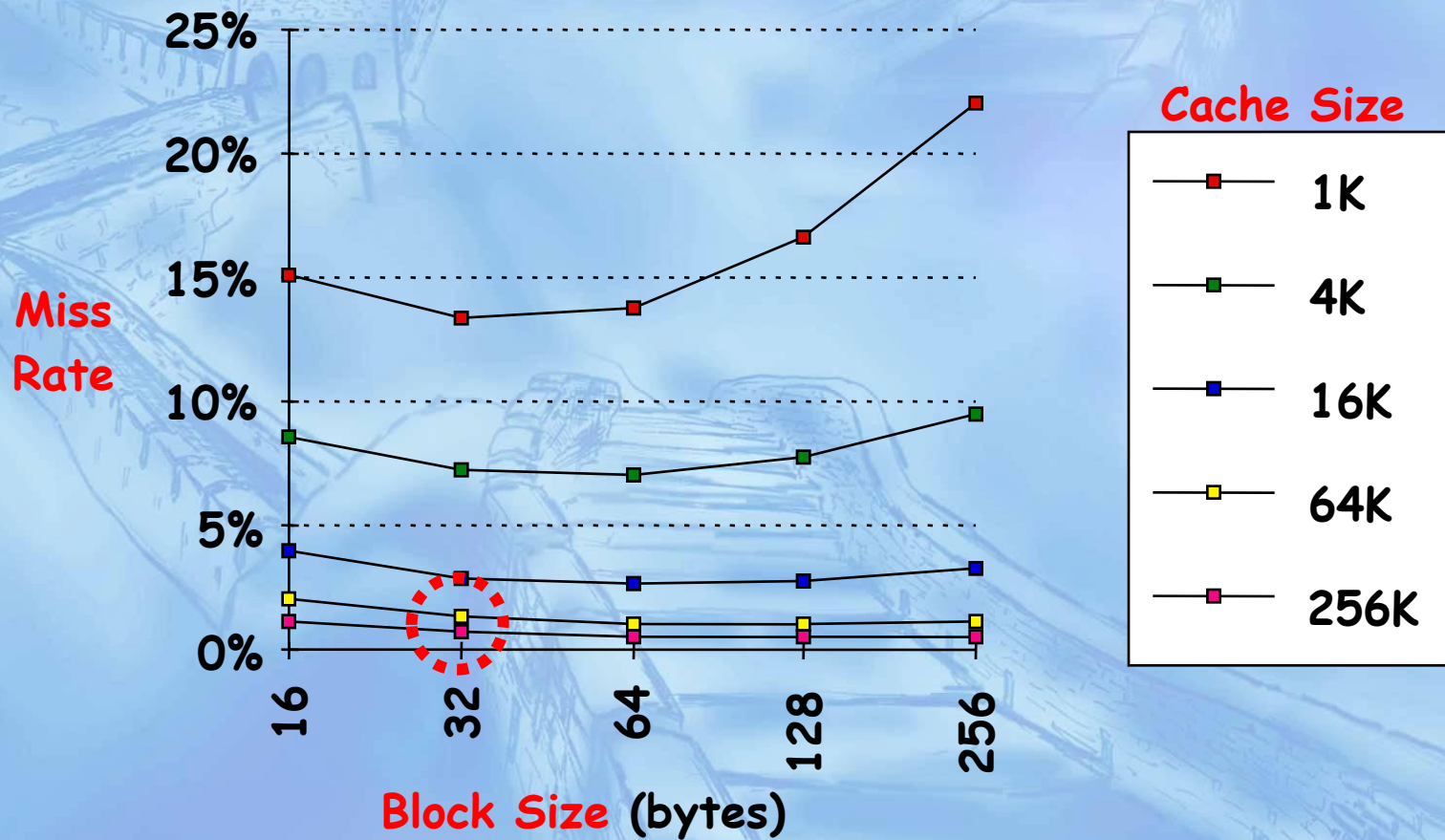
增加Cache块大小

五种不同容量Cache的缺失率与块大小的关系

Block size (B)	Cache size (KB)				
	1	4	16	64	256
16	15.05%	8.57%	3.94%	2.04%	1.09%
32	13.34%	7.24%	2.87%	1.35%	0.70%
64	13.76%	7.00%	2.64%	1.06%	0.51%
128	16.64%	7.78%	2.77%	1.02%	0.49%
256	22.01%	9.51%	3.29%	1.15%	0.49%



图 示





分 析

◆ 增加块大小会先降低后增加缺失率

增加块大小会降低强制缺失率，这是利用了空间局部性原理。但因为它减少了**Cache**中的块数，加重了冲突缺失，如果**Cache**容量较小时，甚至会有容量缺失。

◆ 增加块大小会增加缺失代价

块大小应为多大
AMAT最小？





AMAT与块大小

假设命中时间为1个时钟周期，缺失时系统开销为80个时钟周期，以后每2个时钟周期传送16B，缺失率参见前表。结果显示：**32/64B**是目前**Cache**所通用的块大小。

Block size	Miss penalty	Cache size				
		1K	4K	16K	64K	256K
16	82	13.341	8.027	4.231	2.673	1.894
32	84	12.206	7.082	3.411	2.134	1.588
64	88	13.109	7.160	3.323	1.933	1.449
128	96	16.974	8.469	3.659	1.979	1.470
256	112	25.651	11.651	4.685	2.288	1.549



块大小的选择

块大小的选择取决于较低层存储器的延迟和带宽：低延迟和高带宽存储器使得块大小要大些，因为这样在每次缺失时**Cache**可以获得更多的字节，而缺失代价只有少量的增加；相反，高延迟和低带宽存储器希望块大小要小些，因为较大的块并不能节省多少时间。



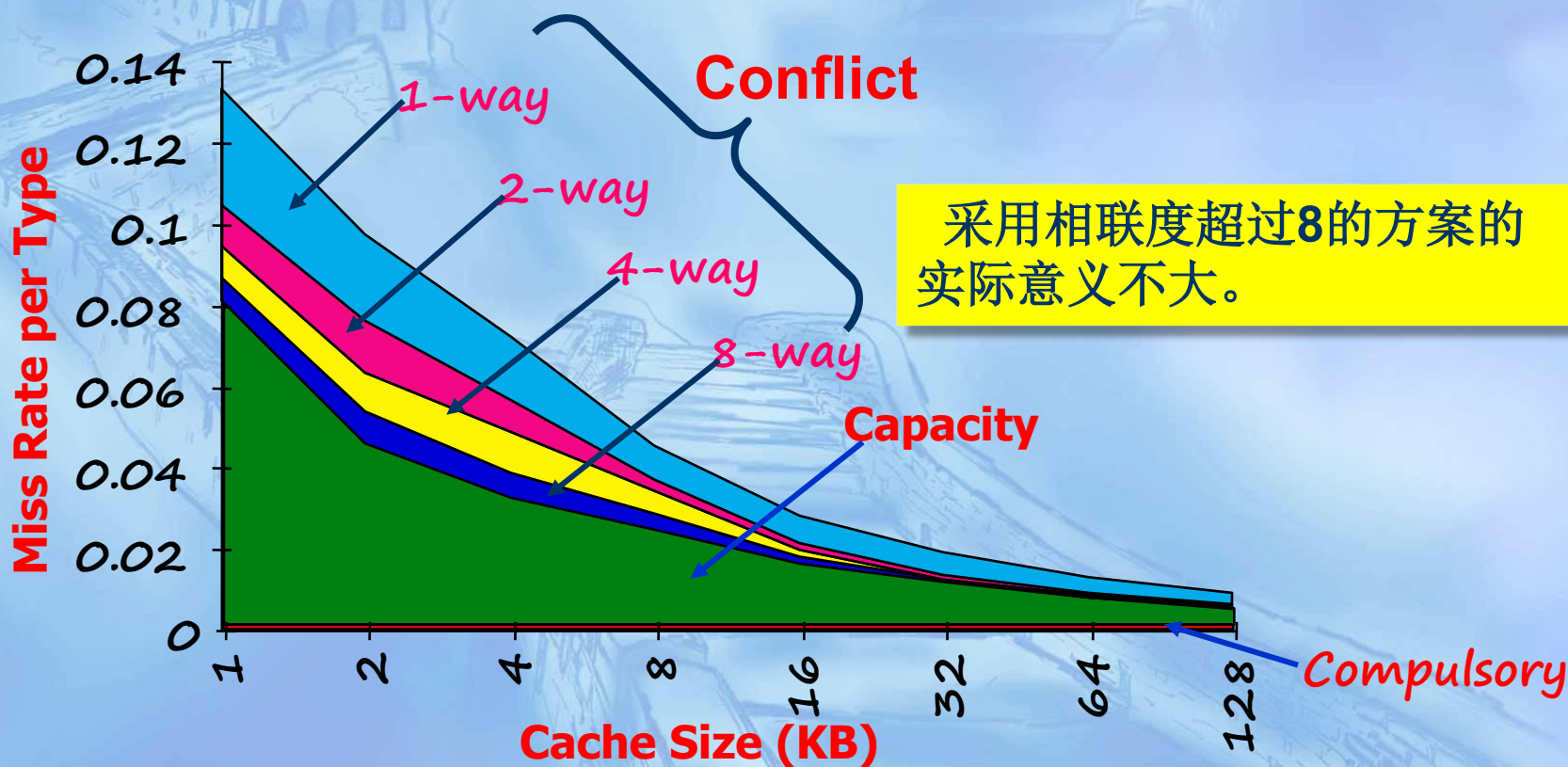
增加Cache容量

增加Cache容量可以降低容量缺失，但增加了命中时间和成本。这种技术在片外Cache中很通用。



增加相联度

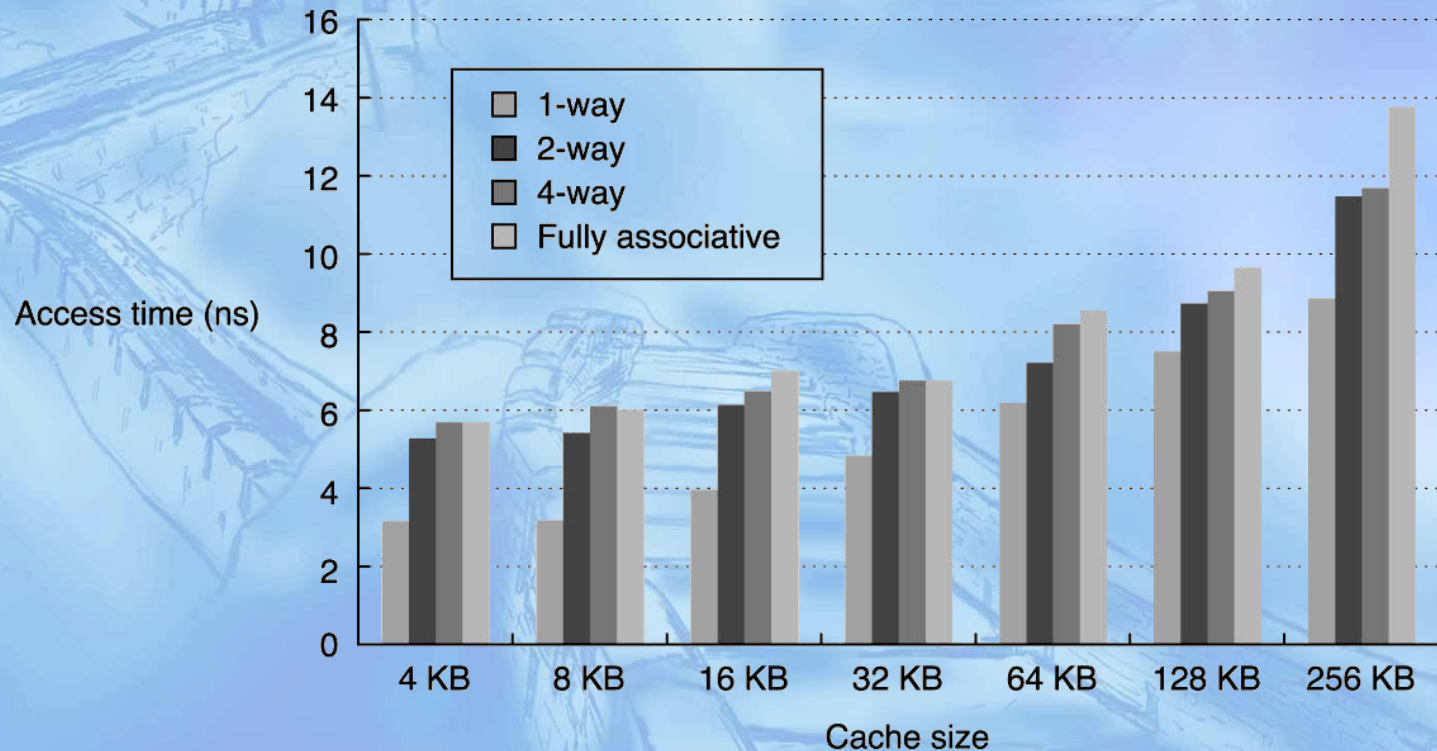
- ◆ 增加相联度会降低缺失率





增加相联度

◆ 增加相联度会增加命中时间





AMAT与相联度

实验结果显示：较高的相联度增加了**AMAT**。

Cache size (KB)	Associativity			
	One-way	Two-way	Four-way	Eight-way
4	3.44	3.25	3.22	3.28
8	2.69	2.58	2.55	2.62
16	2.23	2.40	2.46	2.53
32	2.06	2.30	2.37	2.45
64	1.92	2.14	2.18	2.25
128	1.52	1.84	1.92	2.00
256	1.32	1.66	1.74	1.82
512	1.20	1.55	1.59	1.66



路预测

◆ 思想

在**Cache**中设置一些特殊位来预测下次**Cache**访问中可能会在组中用到的路/块。

◆ 特点

在降低冲突缺失的同时，保持直接映象**Cache**的命中速度。

◆ 例子

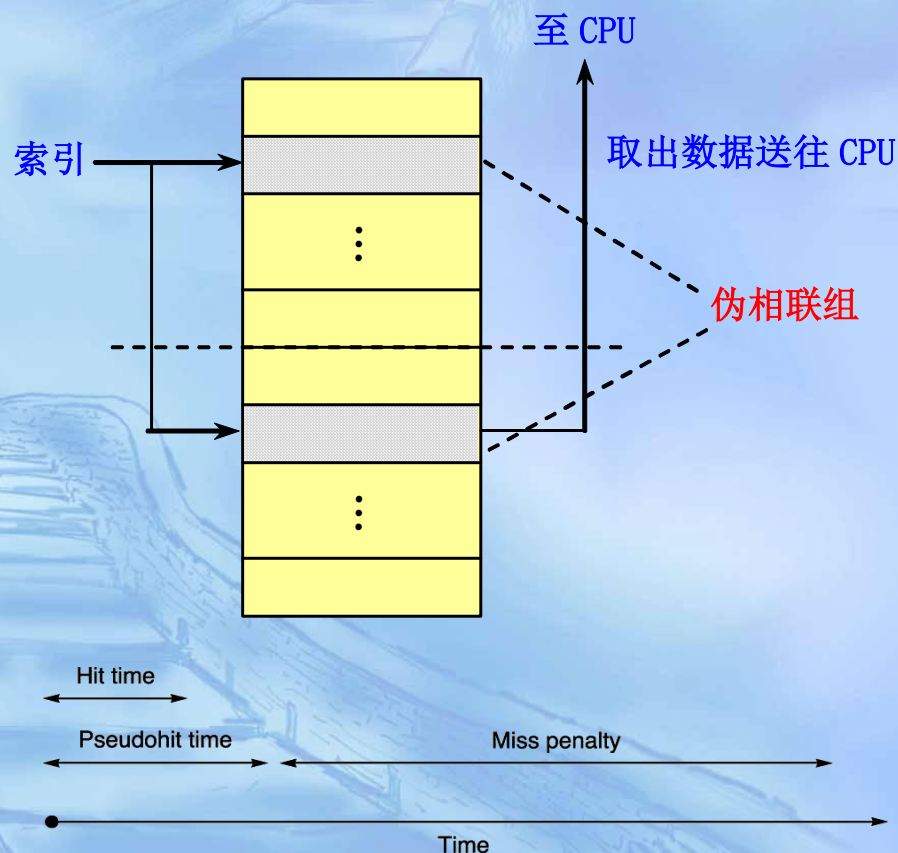
Alpha 21264在2-路组相联指令**Cache**使用路预测，**SPEC95**仿真结果显示：预测正确率超过**85%**。



伪相联Cache

◆ 思想

在逻辑上把直接映象Cache的空间上下平分为两个区。对于任何一次访问，伪相联Cache先按直接映象Cache的方式去处理。若命中，则其访问过程与直接映象Cache的情况一样（命中时间）。若不命中，则再到另一区相应的位置去查找。若找到，则发生了伪命中（伪命中时间），否则就只好访问下一级存储器（缺失代价）。





伪相联Cache

◆ 特点

伪相联Cache的AMAT短，但变化的命中时间会使流水线CPU设计复杂度增加，因此较适合用于离CPU较远的Cache中，例如：L2 Caches。



这两个技术都综合了1-路组相联（直接映象）的命中时间短和2-路组相联的冲突低。



编译优化

◆ 指令缺失性能改善

在不影响正确性的前提下重新安排程序代码，可能会降低冲突缺失，从而降低指令缺失率。例如：**McFarling[1989]**在容量**2KB**、块大小**4B**的直接映象**Cache**中使用本方法将**Cache**的缺失率降低**50%**，若**Cache**容量为**8KB**，则缺失率降低**75%**。



编译优化

◆ 数据缺失性能改善

数据比指令代码在位置上的限制甚至更少，这些变换的目标是尽力提高数据的空间和时间局部性。

下面通过3个例子进行介绍：

循环交换

循环融合

分块



内外循环交换

一些程序带有嵌套循环，它们访问存储器中的数据是非顺序的，简单的交换嵌套循环可以使得代码按照存储顺序来访问数据。

通过提高空间局部性来减少缺失。

/* Before */

```
for (k = 0; k < 100; k = k+1)
  for (j = 0; j < 100; j = j+1)
    for (i = 0; i < 5000; i = i+1)
      x[i][j] = 2 * x[i][j];
```

/* After */

```
for (k = 0; k < 100; k = k+1)
  for (i = 0; i < 5000; i = i+1)
    for (j = 0; j < 100; j = j+1)
      x[i][j] = 2 * x[i][j];
```



循环融合

一些程序有分立的代码段，这些代码按照相同的循环访问相同的数组，对相同的数据进行不同的计算。通过“融合”这些代码到一个循环中，使得装入到Cache中的数据在被替换出来之前可以被重复地利用。



循环融合

/* Before */

```
for (i = 0; i < N; i = i+1)
    for (j = 0; j < N; j = j+1)
        a[i][j] = 1/b[i][j] * c[i][j];
for (i = 0; i < N; i = i+1)
    for (j = 0; j < N; j = j+1)
        d[i][j] = a[i][j] + c[i][j];
```

/* After */

```
for (i = 0; i < N; i = i+1)
    for (j = 0; j < N; j = j+1)
    {
        a[i][j] = 1/b[i][j] * c[i][j];
        d[i][j] = a[i][j] + c[i][j];
    }
```

通过提高时间局部性来减少缺失。



分 块

分块不是对矩阵中的整行/列进行操作，而是对子矩阵或矩阵块进行操作，目标是在调入到**Cache**中的块被替换之前最大限度地利用它。

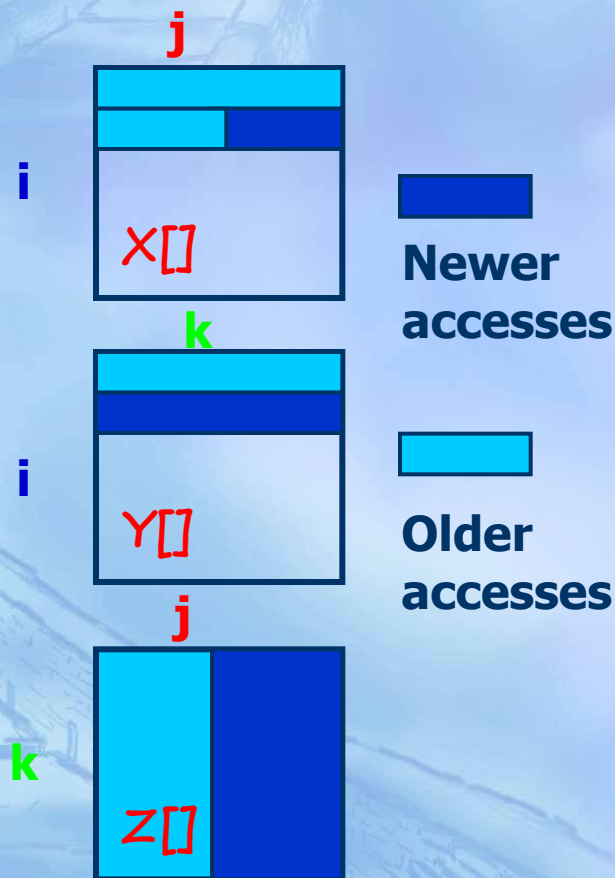


分 块

/ Before */*

```
for (i = 0; i < N; i = i+1)
  for (j = 0; j < N; j = j+1)
  {
    r = 0;
    for (k = 0; k < N; k = k+1)
      r = r + y[i][k]*z[k][j];
    x[i][j] = r;  }
```

i循环一次，要读取矩阵z的所有N×N个元素，对矩阵y一行中的N个元素进行重复访问，对矩阵x一行中的N个元素进行写操作。





分 块

/ After */*

```
for (jj = 0; jj < N; jj = jj+B)
  for (kk = 0; kk < N; kk = kk+B)
    for (i = 0; i < N; i = i+1)
      for (j = jj; j < min(jj+B-1,N); j = j+1)
      {
        r = 0;
        for (k = kk; k < min(kk+B-1,N); k = k+1)
          r = r + y[i][k]*z[k][j];
        x[i][j] = x[i][j] + r;
      }
```

B为分块因子， $B \times B$ 子矩阵都在Cache中。本技术是通过提高空间和时间局部性来减少缺失。



通过并行性降低缺失 代价/缺失率

- ◆ 非阻塞Cache技术
- ◆ 指令和数据硬件预取
- ◆ 编译控制的预取



非阻塞Cache技术

◆ 问题

当Cache失效时，在请求字到达之前，CPU处于等待状态，如何降低CPU的等待时间（缺失代价）？

◆ 解决

非阻塞Cache技术：在Cache失效时仍允许CPU进行其他的命中访问（**允许指令乱序执行**：在CPU等待时有可能执行后面的指令）。即允许“**失效下命中**”。

◆ 优化

- “多重失效下命中”
- “失效下失效”

前提：存储器必须能够处理多个失效

“多重失效下命中” Cache的平均存储器等待时间与阻塞Cache的平均等待时间的比值



	Hit under 1 miss	Hit under 2 miss	Hit under 64 miss
浮点程序 (平均值)	76%	51%	39%
定点程序 (平均值)	81%	78%	78%

- 测试环境：
8KB直接映象Cache，
块大小为32B
- 测试程序：
SPEC92
（14个浮点程序、4个
整数程序）

对整数程序来说，重叠次数对性能提高影响不大，简单的“一次失效下命中”就可以了



指令和数据硬件预取

◆ **思想：**在CPU访问指令和数据之前就将它们预取到Cache或预取到可以比主存访问速度更快的外部缓冲区中。

- 预取通常由Cache之外的硬件完成
- 预取技术建立在利用存储器空闲带宽的基础上，编译器可以帮助减少不必要的预取。

◆ **例子**

AXP21064微处理器在缺失时取两个块：被请求的块和其后续紧挨着的块。被请求的块装入到指令Cache中，而预取的块被装入到指令流缓冲区中。如果某次被请求块在指令流缓冲区中找到，则原Cache请求被取消，块被从流缓冲区中读入，然后下一个预取请求被发出。



硬件预取效果

◆ Joppi的研究结果

- 指令预取 (4 KB, 直接映象Cache, 块大小为16 B)
 - 1个块的指令流缓冲器: 捕获15%~25%的失效
 - 4个块的指令流缓冲器: 捕获50%
 - 16个块的指令流缓冲器: 捕获72%
- 数据预取 (4 KB, 直接映象Cache)
 - 1个数据流缓冲器: 捕获25%的失效
 - 还可以采用多个数据流缓冲器

◆ Palacharla和Kessler的研究结果

- 流缓冲器: 既能预取指令又能预取数据
- 对于两个64 KB四路组相联Cache来说: 8个流缓冲器能捕获50%~70%的失效



编译控制的预取

◆ 思想

利用编译器来插入预取指令，提前发出数据请求，可以将数据预取到寄存器（**寄存器预取**，例如：HP PA-RISC）或将数据预取到Cache（**Cache预取**，例如：MIPS IV、PowerPC、SPARC v.9）。

◆ 提示

- 在预取数据的同时，处理器应能继续执行（即：执行指令和读取数据重叠执行）
- 循环是预取优化的主要对象
 - 失效开销小时：循环体展开1~2次
 - 失效开销大时：循环体展开许多次



降低Cache命中时间

- ◆ 小而简单的Cache
- ◆ 虚拟Cache
- ◆ 流水Cache存取
- ◆ 跟踪Cache



小而简单的Cache

- ◆ **思想：**采用容量小、结构简单的**Cache**来有效降低**Cache**命中时间
 - 硬件越简单，速度就越快；
 - 小容量**Cache**有利于降低**Cache**命中时间，而且应使**Cache**足够小，以便可以与**CPU**一起放在同一块芯片上。
 - 一种折中设计方案：

把**Cache**的标识放在片内，而把**Cache**的数据存储器放在片外。这样既可以实现快速标识检测，又能利用独立的存储芯片来提供更大的容量。



虚拟Cache

按照访问Cache的地址是物理/虚拟地址，可以分为：

◆ 物理Cache

使用物理地址进行访问的传统Cache，其标识存储器中存放的是物理地址，进行地址检测也是用物理地址。

缺点：访问速度很慢（地址转换和访问Cache串行进行）



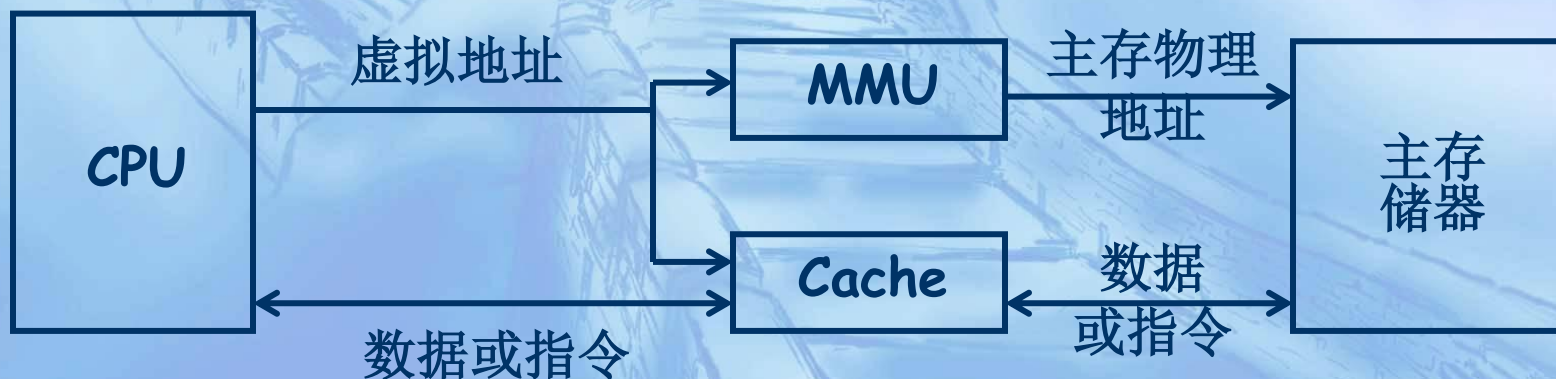


虚拟Cache

◆ 虚拟Cache

使用**虚拟地址**进行访问的**Cache**，其标识存储器中存放的是虚拟地址，进行地址检测也是用虚拟地址。

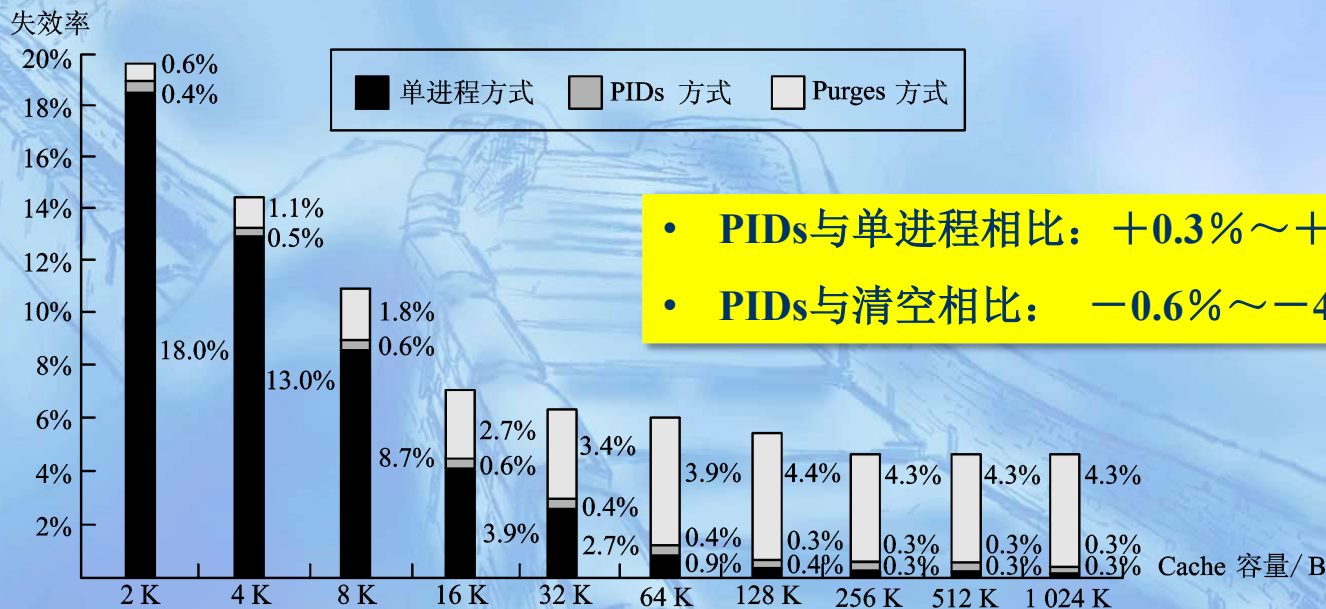
优点：访问速度快（地址转换和访问**Cache**并行进行）





并非都采用虚拟Cache

- ◆ **进程切换时需清空Cache:** 因新进程的虚拟地址可能与原进程相同，但它们所指向的物理空间却是不同的
 - 解决：在地址标识中增加**PID字段** (进程标识符)
 - 下图：采用**PID**所带来的失效率上的改进



- PIDs与单进程相比：+0.3%~+0.6%
- PIDs与清空相比：-0.6%~-4.3%



并非都采用虚拟 Cache

- ◆ **同义和别名：** 同一个物理地址可能采用两种以上不同形式的虚拟地址来访问，这会导致同一个数据在虚拟Cache中存在两个副本
 - **解决：** 反别名法、页着色



虚拟索引 + 物理标识

◆ 思想

直接用虚地址中的页内位移作为访问**Cache**的索引，但标识却是物理地址。**CPU**发出访存请求后，在进行虚-实地址转换的同时，可并行进行标识的读取；在完成地址转换后，再将得到的物理地址与标识进行比较。

◆ **优点**：兼得虚拟**Cache**和物理**Cache**的好处

◆ **局限性**：**Cache**容量受到限制

Cache容量 $\leq$ 页大小 $\times$ 相联度 $=2^{\text{index}} \times$ 块大小 $\times$ 相联度

页地址 地址标识	页内位移	
	索 引	块内位移



三级存储系统

存储系统的组织方式:

- ◆ 两个存储系统的组织方式
- ◆ 一个存储系统的组织方式
- ◆ 全Cache存储系统



两个存储系统的组织方式

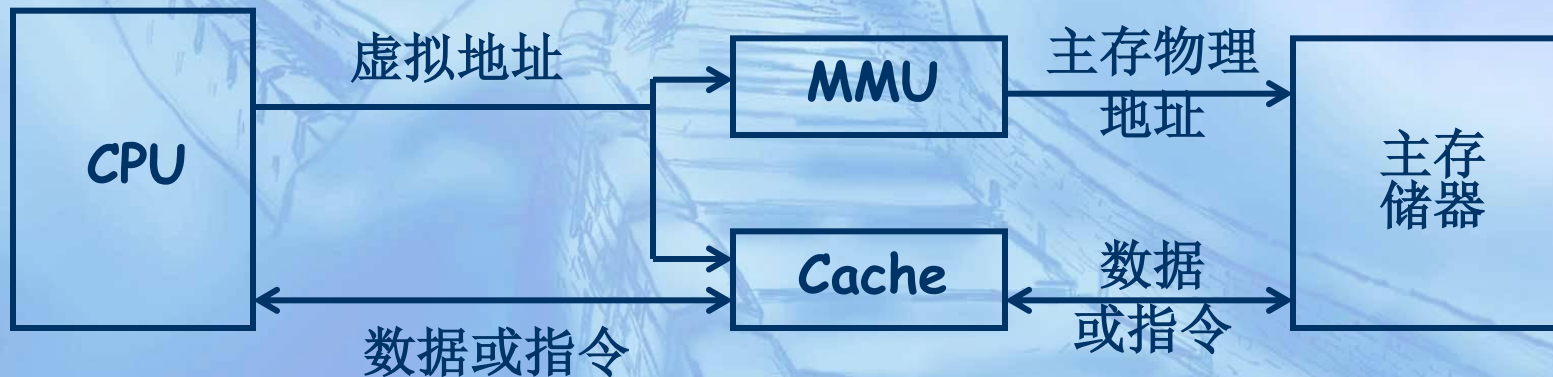
有“Cache-主存”和“主存-磁盘”两个独立的存储系统，这种结构也称为**物理地址Cache存储系统**。目前的大部分处理机均采用这种两级存储系统。





一个存储系统的组织方式

将Cache、主存和磁盘组织在一起构成一个“Cache-主存-磁盘”存储系统，这种结构也称为**虚拟地址Cache存储系统**。如Intel公司的i860等处理机采用这种组织方式。





全Cache存储系统

没有主存储器，只用**Cache**和磁盘构成“**Cache—磁盘**”存储系统。例如：多处理机中的全**Cache**存储系统。